

GIỚI HẠN VÀ MIỀN CHUYÊN



TÓM TẮT

Phần D trả lời hai câu hỏi mà phần C cố tình gác lại: “khi nào thì nên ngừng tìm lời giải nhanh hơn” và “những miền nào có bộ công cụ riêng mà kinh nghiệm chung không chuyển sang được”. Chương 21 là số học và đại số tính toán, chương 22 là hình học tính toán — hai miền mà cạm bẫy chủ yếu không nằm ở ý tưởng mà ở số học hữu hạn. Chương 23 là độ khó và quy dẫn: cách nhận ra một bài là NP-khó và cách sống chung với nó. Chương 24 là cận dưới và các mô hình tính toán khác — luồng dữ liệu, trực tuyến, song song, bộ nhớ ngoài. Nếu chỉ nhớ một điều: biết một bài là NP-khó không phải là thất bại, đó là thông tin đắt giá nhất bạn có thể có trước khi bỏ ra ba ngày tìm lời giải đa thức không tồn tại.

Ba chương đầu của phần này có một điểm chung dễ bị bỏ qua: chúng đều nói về chỗ mà *mô hình lý tưởng gặp thực tế*. Trong chương 21, mô hình lý tưởng nói “cộng hai số mất $O(1)$ ”, còn thực tế nói số nguyên có 64 bit và phép nhân tràn im lặng. Trong chương 22, mô hình lý tưởng nói “ba điểm thẳng hàng hoặc không”, còn thực tế nói số thực dấu phẩy động không có khái niệm bằng nhau. Trong chương 23, mô hình lý tưởng nói “có thuật toán đa thức hoặc không”, còn thực tế nói bài NP-khó vẫn được giải hàng ngày trong công nghiệp bằng heuristic và bộ giải SAT.

Chương 24 đứng riêng vì nó phá bỏ giả định lớn nhất của cả quyển sách: giả định rằng dữ liệu nằm vừa trong bộ nhớ, đọc được nhiều lần, và máy chỉ có một lỗi. Khi giả định đó sai — dữ liệu chảy qua một lần, quyết định phải ra ngay khi chưa thấy tương lai, hoặc đĩa chậm hơn RAM hàng nghìn lần — thì bảng xếp hạng thuật toán thay đổi hoàn toàn. Đây cũng là chương gần với công việc kỹ thuật thật nhất trong cả phần D.

Bản đồ phần D — Giới hạn và miền chuyên

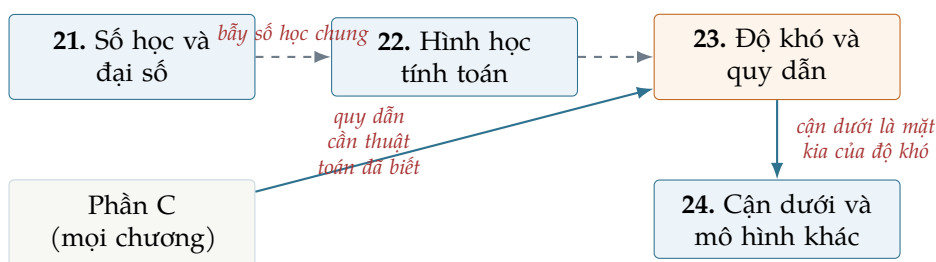
TÓM TẮT

Bốn chương, hai vai trò. Chương 21 và 22 là hai *miền chuyên* có bấy số học riêng. Chương 23 và 24 là hai cách nói về *giới hạn*: giới hạn do bản chất bài toán (NP-khó) và giới hạn do mô hình máy (bộ nhớ, một lượt đọc, không biết tương lai). Chương 23 là chương nên đọc kỹ nhất của phần này vì nó thay đổi cách bạn phân bổ thời gian khi gặp bài lạ.

A. Phần này có gì

- **21-số-học-và-dai-so** — ước chung lớn nhất, số học modulo, nghịch đảo, sàng, phân tích thừa số, định lý phần dư Trung Hoa, lũy thừa ma trận, FFT/NTT, đại số tuyến tính trên $\mathbb{F}_2$.
- **22-hình-học-tính-toan** — vị từ định hướng, bao lồi, quét đường, giao đoạn thẳng, cặp điểm gần nhất, và vấn đề bền vững số học.
- **23-do-kho-và-quy-dan** — P, NP, NP-đầy đủ, cách dựng một quy dẫn, danh mục bài NP-khó hay gặp, và bốn cách sống chung: xấp xỉ, tham số hoá, heuristic, bộ giải.
- **24-can-duoi-và-mô-hình-khac** — cận dưới cây quyết định, lập luận đối thủ, thuật toán luồng dữ liệu, thuật toán trực tuyến và phân tích cạnh tranh, mô hình bộ nhớ ngoài và cache-oblivious, song song.

B. Phụ thuộc và thứ tự đọc



Hình 1. Phần D chủ yếu phụ thuộc vào phần C: muốn quy dẫn bài mới về một bài NP-đầy đủ, trước hết phải biết kho bài toán chuẩn. Chương 23 (tô hồ phách) là chương cốt lõi của phần.

C. Cốt lõi hay tham khảo

Cốt lõi: số học modulo, lũy thừa nhanh và nghịch đảo (21); vị từ định hướng và bao lồi (22); toàn bộ phần nhận diện NP-khó và bốn cách đối phó (23). **Tham khảo:** FFT/NTT ở mức cài đặt (21); quét đường cho giao đoạn thẳng (22); chứng minh Cook–Levin (23); toàn bộ chương 24 trừ mục phân cấp bộ nhớ, vốn đã xuất hiện ở chương 3.

Ở D/22, dùng bộ hình chạm đầu mút, chồng đoạn và tách rời để kiểm quy ước giao trước khi cài; phép quay không loại bỏ thẳng hàng.

Số học và đại số tính toán (Number Theory and Algebra)

PHỤ THUỘC A/05 cho số học modulo cơ bản; C/14 cho chia để trị — FFT là ứng dụng sâu nhất của nó; C/20 cho kiểm tra nguyên tố xác suất.

TÓM TẮT

Miền này có một đặc điểm riêng: cạm bẫy chủ yếu không nằm ở ý tưởng mà ở *số học hữu hạn* — tràn số, phép chia không tồn tại, và sự khác biệt giữa số nguyên toán học với số nguyên 64 bit. Chương đi từ thuật toán Euclid và nghịch đảo modulo, qua sàng và phân tích thừa số, tới định lý phần dư Trung Hoa, lũy thừa ma trận, và biến đổi Fourier nhanh. Xuyên suốt là một cách nhìn thống nhất: rất nhiều kỹ thuật ở đây là “đổi biểu diễn để phép toán khó trở nên dễ”, đúng mẫu tư duy đã gặp ở tổng tiền tố (B/06). Nếu chỉ nhớ một điều: mọi lỗi số học trong thi đấu quy về ba nguyên nhân — tràn khi nhân, chia mà không có nghịch đảo, và kết quả âm sau khi lấy dư.

KIẾN THỨC CẦN CÓ TRƯỚC

- **ước chung lớn nhất** — số lớn nhất chia hết cả hai; tính bằng thuật toán Euclid trong $O(\log \min(a, b))$.
- **Euclid mở rộng** — ngoài $\gcd(a, b)$ còn cho hai số x, y với $ax + by = \gcd(a, b)$; đây là cách tính nghịch đảo modulo.
- **nghịch đảo modulo** — số a^{-1} với $a \cdot a^{-1} \equiv 1 \pmod{m}$; tồn tại khi và chỉ khi $\gcd(a, m) = 1$. Thay cho phép chia.
- **sàng Eratosthenes** — đánh dấu bội của từng số nguyên tố để tìm mọi số nguyên tố tới n trong $O(n \log \log n)$.
- **định lý phần dư Trung Hoa** — biết phần dư của x theo nhiều modulo đôi một nguyên tố cùng nhau thì xác định được x theo tích của chúng.
- **biến đổi Fourier nhanh** — thuật toán $O(n \log n)$ tính tích chập hai dãy; ý tưởng là đổi sang biểu diễn mà tích chập trở thành phép nhân từng điểm.

CÂU HỎI MỞ ĐẦU

- Q1. Thuật toán Euclid ra đời hơn hai nghìn năm trước và vẫn được dùng nguyên vẹn. Điều gì làm nó bền tới vậy?
- Q2. Vì sao trong số học modulo, phép chia lại phải thay bằng phép nhân với nghịch đảo, và vì sao nghịch đảo không phải lúc nào cũng tồn tại?
- Q3. Nhân hai đa thức bậc n tốn $O(n^2)$ theo cách thông thường. FFT xuống $O(n \log n)$ bằng cách nào, và ý tưởng đó có họ hàng gì với tổng tiền tố?
- Q4. Vì sao các bài đếm hay yêu cầu modulo 998 244 353 thay vì $10^9 + 7$?
- Q5. Bài toán phân tích một số thành thừa số nguyên tố khó tới mức nào, và vì sao độ khó đó lại là nền của mã hoá hiện đại?

Miền số học có một đặc điểm khiến nó khác các chương trước: phần lớn lỗi ở đây không phải lỗi thuật toán mà là lỗi *số học hữu hạn*. Bạn viết đúng công thức toán học, nhưng máy tính làm

việc với số nguyên 64 bit và với phép chia lấy dư có dấu, và kết quả sai ở những chỗ mà toán học không hề cảnh báo.

Vì vậy chương này sẽ nhấn mạnh ba thứ song song: ý tưởng toán học, cách nó trở thành thuật toán, và *chỗ nó hỏng trên máy thật*. Cách sắp xếp đó phản ánh đúng thứ tự mà bạn sẽ gặp vấn đề trong thực tế.

1.1 Euclid: thuật toán cổ nhất còn dùng

Thuật toán Euclid tính $\gcd(a, b)$ bằng cách thay (a, b) bởi $(b, a \bmod b)$ cho tới khi số thứ hai bằng 0. Tính đúng dựa trên một bất biến rất gọn: $\gcd(a, b) = \gcd(b, a \bmod b)$. Tính đúng dựa trên một đại lượng giảm: số thứ hai giảm nghiêm ngặt. Đây là ví dụ mẫu mực cho bộ đôi công cụ ở chương A/04, và cũng là lý do thuật toán này thường được dùng làm ví dụ đầu tiên khi dạy chứng minh.

Chi phí là $O(\log \min(a, b))$, và trường hợp xấu nhất xảy ra đúng với các số Fibonacci liên tiếp — một liên hệ đẹp giữa lý thuyết số và tổ hợp.

Euclid mở rộng cho thêm hai hệ số x, y thoả $ax + by = \gcd(a, b)$. Ứng dụng quan trọng nhất là tính nghịch đảo modulo: nếu $\gcd(a, m) = 1$ thì $ax + my = 1$, tức $ax \equiv 1 \pmod{m}$, nên x chính là a^{-1} . Đây là câu trả lời cho câu hỏi mở đầu thứ hai: trong số học modulo không có phép chia; “chia cho a ” được định nghĩa là “nhân với nghịch đảo của a ”, và nghịch đảo chỉ tồn tại khi a và m nguyên tố cùng nhau.

Hệ quả thực hành rất cụ thể: khi m là số nguyên tố thì mọi số khác 0 đều có nghịch đảo, nên mọi công thức có phân số đều tính được. Đó chính là lý do đề bài luôn chọn modulo nguyên tố.

1.2 Sàng, phân tích thừa số, và ranh giới của cái dễ

Sàng Eratosthenes tìm mọi số nguyên tố tới n bằng cách đánh dấu bội của từng số nguyên tố. Chi phí là $O(n \log \log n)$ — và con số $\log \log$ này đến từ tổng $\sum_p n/p$ trên các số nguyên tố, một kết quả của lý thuyết số mà chương A/05 đã nhắc.

Hai biến thể đáng biết: sàng lưu *ước nguyên tố nhỏ nhất* của mỗi số, cho phép phân tích thừa số bất kỳ số nào $\leq n$ trong $O(\log n)$; và *sàng tuyến tính*, đảm bảo mỗi hợp số bị đánh dấu đúng một lần, cho $O(n)$ nhưng hằng số không nhỏ hơn nhiều nên hiếm khi cần.

Phân tích một số lớn thì hoàn toàn khác. Thử chia tới $\sqrt{n}$ là $O(\sqrt{n})$ — chấp nhận được tới 10^{12} nhưng không hơn. Thuật toán Pollard rho, một thuật toán ngẫu nhiên đẹp, cho kỳ vọng $O(n^{1/4})$ và xử lý được số 64 bit. Nhưng với số hàng trăm chữ số thì không thuật toán nào đã biết chạy nổi trong thời gian hợp lý.

Ranh giới đó chính là điều đáng nhớ nhất mục này, và nó là câu trả lời cho câu hỏi mở đầu thứ năm. Kiểm tra một số có nguyên tố hay không thì *dễ* (Miller–Rabin, C/20); phân tích nó ra thừa số thì *khó*. Sự bất đối xứng giữa hai bài toán tưởng như anh em ấy là nền của mã hoá khoá công khai: ai cũng nhân được hai số nguyên tố lớn với nhau, nhưng không ai tách ngược lại được.

CẠM BẮY

Ba lỗi số học chiếm gần hết các lần sai trong bài modulo. *Trên khi nhân*: với $m \approx 10^9$, tích hai phần dư đã cỡ 10^{18} , sát giới hạn số nguyên 64 bit có dấu — và nếu $m \approx 10^{18}$ thì bắt buộc phải dùng kỹ thuật nhân đặc biệt. *Kết quả âm*: trong nhiều ngôn ngữ, phép lấy dư của số âm trả về số âm, nên sau phép trừ phải cộng thêm m rồi lấy dư lần nữa. *Chia mà không*

kiểm nghiệm đảo: nếu modulo không nguyên tố và số chia không nguyên tố cùng nhau với nó thì nghiệm đảo không tồn tại, và công thức của bạn vô nghĩa chứ không chỉ sai số.

1.3 “Kiểm tra thì dễ” – dễ tới mức nào, và từ khi nào

Câu “kiểm tra một số có nguyên tố hay không thì dễ” ở mục trên xứng đáng được nói chính xác hơn, vì đằng sau nó là một trong những kết quả đẹp nhất của lý thuyết tính toán.

Miller–Rabin là thuật toán *ngẫu nhiên*: nó chỉ sai với xác suất nhỏ tùy ý, và trong thực tế thì thế là quá đủ. Nhưng câu hỏi lý thuyết vẫn để ngỏ suốt nhiều thập kỷ: có thuật toán *tất định*, thời gian đa thức, không cần giả thuyết chưa chứng minh nào, để kiểm tra tính nguyên tố hay không? Bản tất định của Miller chạy đa thức nhưng phải dựa vào một giả thuyết lớn của lý thuyết số, nên nó không tính là câu trả lời.

Câu trả lời tới năm 2002, từ Agrawal cùng hai sinh viên của ông là Kayal và Saxena, và được đăng trên *Annals of Mathematics* hai năm sau[1] với một nhan đề gọn tới mức nổi tiếng: “PRIMES is in P”. Thuật toán không dài, không dùng công cụ nặng, và chứng minh vừa trong mười trang. Ý nghĩa của nó với chương này là làm sắc lại đúng sự bất đối xứng đã nói ở mục 2: *kiểm tra* tính nguyên tố nằm trong P một cách vô điều kiện, trong khi *phân tích thừa số* vẫn chưa ai biết cách làm trong thời gian đa thức. Toàn bộ mã hoá khoá công khai sống trên khe hở giữa hai câu ấy.

Và như mọi lần trong quyển sách này, phải kèm câu thứ hai: không ai dùng AKS trong thực tế. Số mũ của nó lớn tới mức Miller–Rabin lặp vài chục vòng nhanh hơn nhiều bậc, với xác suất sai nhỏ hơn xác suất máy tính lỗi phần cứng. Đây là cùng bài học của C/14: kết quả trả lời câu hỏi “có thể hay không”, chứ không phải câu hỏi “nên cài gì”. Người học nên tách bạch hai câu hỏi đó ngay từ đầu, vì trộn chúng là nguồn của cả sự hoài nghi vô lý với lý thuyết lẫn sự sùng bái vô lý với các cặn mới.

1.4 Định lý phần dư Trung Hoa: ghép các mảnh thông tin

Nếu biết $x \equiv a_1 \pmod{m_1}$ và $x \equiv a_2 \pmod{m_2}$ với m_1, m_2 nguyên tố cùng nhau, thì tồn tại duy nhất một x modulo $m_1 m_2$. Ý nghĩa thực dụng: bạn có thể *tách* một phép tính lớn thành nhiều phép tính nhỏ theo các modulo nhỏ, làm độc lập, rồi ghép lại.

Ba ứng dụng hay gặp. Thứ nhất, tránh tràn: thay vì làm việc với modulo 10^{18} , làm hai lần với hai modulo cỡ 10^9 rồi ghép. Thứ hai, tăng độ tin cậy của băm chuỗi (B/08): hai modulo cho xác suất va chạm bằng tích hai xác suất. Thứ ba, trong biến đổi Fourier trên số nguyên khi modulo của bài không có tính chất cần thiết: làm với vài modulo thân thiện rồi ghép.

Nhìn ở mức trừu tượng hơn, đây lại là mẫu quen thuộc: *đổi biểu diễn để phép toán trở nên dễ*. Ta chuyển một số lớn thành bộ phần dư của nó, làm việc trong biểu diễn đó, rồi chuyển ngược. Cùng ý tưởng với tổng tiền tố và mảng hiệu ở B/06, và với FFT ở mục sau.

1.5 Luỹ thừa nhanh và luỹ thừa ma trận

Tính $a^n \bmod m$ bằng cách nhân n lần là $O(n)$; bằng bình phương liên tiếp là $O(\log n)$. Ý tưởng: $a^n = (a^{n/2})^2$ nếu n chẵn, và $a \cdot a^{n-1}$ nếu lẻ — chia để trị ở dạng đơn giản nhất.

Cùng kỹ thuật áp cho ma trận, và đó là công cụ giải các bài “tính trạng thái sau 10^{18} bước”. Điều kiện, như đã nói ở A/05, là quan hệ truy hồi phải *tuyến tính*. Khi đó $v_n = A^n v_0$ và A^n tính được trong $O(d^3 \log n)$ với d là kích thước ma trận.

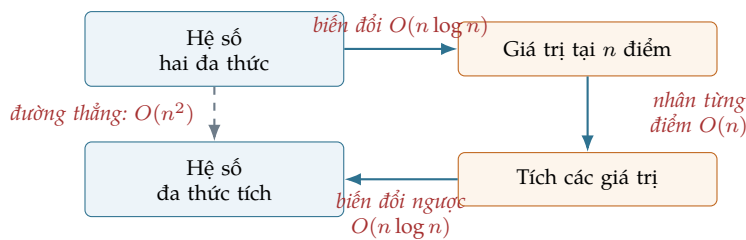
Ba mẹo mở rộng đáng biết. Nếu truy hồi có thêm hằng số cộng, hãy thêm một chiều mang giá trị 1 vào vector trạng thái. Nếu cần *tổng* của dãy, thêm một chiều tích lũy. Và nếu bài hỏi số đường đi độ dài k trong đồ thị, lũy thừa ma trận kể đếm đúng thứ đó — một liên hệ đẹp giữa đại số và đồ thị (C/17).

1.6 Biến đổi Fourier nhanh: đổi miền để nhân trở thành dễ

Bài toán: cho hai đa thức bậc n , tính tích. Cách trường học tốn $O(n^2)$. Karatsuba (C/14) cho $O(n^{1,585})$. FFT cho $O(n \log n)$.

Ý tưởng gồm ba bước, và bước giữa mới là chỗ đáng nhớ. Một đa thức bậc n được xác định hoàn toàn bởi giá trị của nó tại $n + 1$ điểm. Trong *biểu diễn theo giá trị*, phép nhân hai đa thức trở thành phép nhân từng điểm — chỉ $O(n)$. Vậy thuật toán là: đổi cả hai đa thức sang biểu diễn giá trị, nhân từng điểm, rồi đổi ngược về biểu diễn hệ số. Phần khó là làm hai phép đổi đó nhanh, và FFT làm được nhờ chọn các điểm rất đặc biệt — các căn bậc n của đơn vị — khiến phép đổi có cấu trúc chia để trị.

Đây là câu trả lời cho câu hỏi mở đầu thứ ba, và nó cho thấy rõ mẫu tư duy chung của cả chương: *phép toán khó ở biểu diễn này lại dễ ở biểu diễn kia; hãy đi đường vòng*. Bạn đã gặp mẫu này ba lần — tổng tiền tố và mảng hiệu, định lý phần dư Trung Hoa, và bây giờ là FFT.



Hình 1.1. Mẫu “đi đường vòng” của cả chương. Phép nhân tốn $O(n^2)$ ở biểu diễn hệ số nhưng chỉ $O(n)$ ở biểu diễn giá trị, nên ta đổi biểu diễn, làm việc, rồi đổi ngược. Cùng mẫu xuất hiện ở tổng tiền tố và mảng hiệu (B/06) và ở định lý phần dư Trung Hoa.

Biến đổi trên số nguyên (NTT) thay các căn phức bằng các căn của đơn vị trong số học modulo. Điều kiện là modulo phải có căn nguyên thủy với bậc là lũy thừa đủ lớn của 2 — và đó chính là lý do 998 244 353 được ưa dùng, vì nó bằng $119 \cdot 2^{23} + 1$. Đây là câu trả lời cho câu hỏi mở đầu thứ tư: modulo đó không phải chọn tùy tiện mà được chọn vì tính chất đại số của nó.

1.7 Đại số trên $\mathbb{F}_2$: khi cộng là XOR

Khi làm việc với các bit, phép cộng modulo 2 chính là XOR, và cả bộ máy đại số tuyến tính vẫn dùng được — chỉ khác là mọi phép toán chạy trên bit nên rất nhanh.

Ứng dụng chính là *cơ sở tuyến tính* của một tập số: duy trì một tập tối thiểu các số sao cho mọi XOR của tập gốc đều biểu diễn được. Với cơ sở đó, ta trả lời được “số này có là XOR của một tập con không”, “XOR lớn nhất có thể là bao nhiêu”, và “có bao nhiêu giá trị XOR phân biệt” — tất cả trong $O(\log C)$ mỗi thao tác.

Cùng khung áp cho giải hệ phương trình tuyến tính trên $\mathbb{F}_2$, thứ xuất hiện trong các bài về đèn và công tắc, và trong lý thuyết mã sửa lỗi.

1.8 Gốc rễ: từ Euclid tới RSA

Chương này có gốc rễ cổ nhất trong cả quyển sách. Thuật toán Euclid xuất hiện trong bộ *Cơ sở* khoảng năm 300 trước Công nguyên, và nó thường được gọi là thuật toán lâu đời nhất còn được dùng nguyên vẹn tới ngày nay — không phải một phiên bản cải tiến, mà đúng thủ tục đó. Sàng Eratosthenes cũng thuộc thời kỳ đó. Điều đáng suy nghĩ: cả hai đều là những thuật toán mà tính đúng dựa trên một bất biến rất gọn, và có lẽ chính sự gọn ấy là lý do chúng sống lâu.

Định lý phần dư Trung Hoa có tên như vậy vì dạng sớm nhất của nó xuất hiện trong một sách toán Trung Quốc khoảng thế kỷ 3–5, dưới hình thức một bài đố: tìm số mà chia 3 dư 2, chia 5 dư 3, chia 7 dư 2. Việc một bài đố dân gian trở thành công cụ tính toán hiện đại là một mẫu lặp lại trong lịch sử toán học.

Bước ngoặt lớn nhất của miền này thì rất gần đây và có tính bước ngoặt thật sự. Suốt hàng thế kỷ, lý thuyết số được coi là ngành toán “thuần túy nhất”, không có ứng dụng — một nhà toán học nổi tiếng đầu thế kỷ 20 còn tự hào về điều đó. Rồi năm 1976–78, ý tưởng mã hoá khoá công khai ra đời, và hệ RSA dựa trực tiếp lên sự bất đối xứng ở mục 2: nhân hai số nguyên tố lớn thì dễ, tách ngược thì khó. Chỉ trong vài năm, những kết quả về số nguyên tố và số học modulo từ chỗ là trò chơi trí tuệ trở thành hạ tầng của thương mại điện tử và của mọi kết nối an toàn trên Internet.

Bài học từ câu chuyện đó vượt ra ngoài số học: *một bài toán được chứng minh là khó có thể trở thành tài nguyên quý*. Trong chương [D/23](#) ta sẽ gặp lại ý này ở dạng tổng quát — độ khó không chỉ là chướng ngại, đôi khi nó là thứ ta cần.

Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
~300 TCN — Euclid	Tìm ước chung lớn nhất của hai đoạn thẳng (đo lường hình học).	Thuật toán cổ nhất còn dùng nguyên vẹn; tính đúng dựa trên một bất biến, tính dùng dựa trên một đại lượng giảm.
~200 TCN — Eratosthenes	Liệt kê số nguyên tố.	Sàng; vẫn là cách nhanh nhất để tìm mọi số nguyên tố trong một khoảng.
Thế kỷ 3–5 — toán học Trung Quốc	Bài đố: tìm số biết các phần dư.	Định lý phần dư Trung Hoa; nay dùng để tách phép tính lớn thành nhiều phép tính nhỏ.
1640–1801 — Fermat, Euler, Gauss	Hệ thống hoá tính chất của số nguyên tố và đồng dư.	Định lý Fermat nhỏ (nền của cả nghịch đảo modulo lẫn kiểm tra nguyên tố) và ngôn ngữ đồng dư hiện đại.
1965 — Cooley & Tukey	Xử lý tín hiệu địa chấn quy mô lớn.	FFT; ý tưởng “đổi miền để tích chập thành phép nhân từng điểm” trở thành công cụ phổ quát.
1976–78 — mã hoá khoá công khai, RSA	Trao đổi khoá an toàn qua kênh công khai.	Lý thuyết số từ ngành “thuần túy nhất” trở thành hạ tầng của Internet; sự khó của bài phân tích thừa số trở thành tài nguyên.
1975–76 — kiểm tra nguyên tố xác suất	Cần kiểm nguyên tố số hàng trăm chữ số để sinh khoá.	Miller–Rabin; ngẫu nhiên hoá trở thành công cụ sản xuất, không phải giải pháp tạm ($C/20$).

1.9 Liên hệ ngang

Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Mật mã	RSA, trao đổi khóa, chữ ký số	Dùng đúng các công cụ của chương: lũy thừa modulo, nghịch đảo, kiểm tra nguyên tố. Khác biệt quyết định: ở đó ta dựa vào việc bài toán ngược <i>không</i> giải được nhanh — độ khó là tính năng, không phải lỗi.
Mã sửa lỗi	Reed–Solomon trong mã QR, đĩa quang, truyền tin vệ tinh	Đại số đa thức trên trường hữu hạn — cùng bộ máy với FFT/NTT. Ý tưởng chung: một đa thức bậc k được xác định bởi $k + 1$ điểm, nên gửi dư điểm thì mất vài điểm vẫn khôi phục được.
Xử lý tín hiệu	Tích chập, lọc, phổ tần số	FFT là công cụ nền. Trong thuật toán ta dùng nó để nhân đa thức; trong xử lý tín hiệu, cùng phép biến đổi cho ý nghĩa vật lý là phân tích tần số.
Đồ hoạ và học sâu	Tích chập trong mạng nơ-ron, lọc ảnh	Phép tích chập giống hệt phép nhân đa thức. Trong thực tế, thư viện chọn giữa tích chập trực tiếp và FFT tùy kích thước hạt nhân — đúng bài toán ngưỡng hoà vốn ở $C/14$.
Truyền thông và lưu trữ	Tổng kiểm tra CRC	Số học đa thức trên $\mathbb{F}_2$ — cùng đại số với mục 6. Đây là nơi phép XOR làm việc như phép cộng và toàn bộ lý thuyết vẫn đúng.
Hệ thống nhúng, cảm biến	Số học dấu phẩy tĩnh, tránh chia biến	Cùng mối quan tâm với mục 2: phép chia đắt và có thể mất chính xác, nên người ta thay bằng nhân với nghịch đảo đã tính trước — ý tưởng giống hệt nghịch đảo modulo, chỉ ở miền khác.
Tính toán khoa học	Số học chính xác tùy ý, hệ đại số máy tính	Khi kết quả phải chính xác tuyệt đối — ví dụ kiểm chứng một chứng minh — người ta dùng số nguyên lớn, và mọi thuật toán ở đây phải tính lại chi phí theo <i>số bit</i> thay vì coi mỗi phép toán là $O(1)$ ($A/03$).

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì nó dựa trên một bất biến rất gọn — $\gcd(a, b) = \gcd(b, a \bmod b)$ — và một đại lượng giảm hiển nhiên, nên nó vừa đúng vừa dừng mà không cần giả định gì về dữ liệu. Nó cũng không có tham số nào phải tinh chỉnh, không có trường hợp xấu phụ thuộc dữ liệu, và chi phí $O(\log \min(a, b))$ là gần như tối ưu. Những thuật toán sống lâu thường có chung đặc điểm đó: ít giả định, ít tham số, chứng minh ngắn.
- A2. Vì trong tập các phần dư modulo m , phép chia không được định nghĩa như trên số hữu tỉ — ta chỉ có cộng, trừ, nhân. “Chia cho a ” được định nghĩa lại là “nhân với số a^{-1} sao cho $a \cdot a^{-1} \equiv 1$ ”. Số đó tồn tại khi và chỉ khi $\gcd(a, m) = 1$; nếu a và m có ước chung thì mọi bội của a đều nằm trong một tập con thực sự của các phần dư và không bao giờ chạm tới 1. Đây là lý do modulo nguyên tố được ưa chuộng: khi đó mọi số khác 0 đều nghịch đảo được.
- A3. Bằng cách đối biểu diễn. Một đa thức bậc n được xác định bởi giá trị tại $n + 1$ điểm, và trong biểu diễn theo giá trị thì nhân hai đa thức chỉ là nhân từng điểm, tốn $O(n)$. Toàn bộ việc còn lại là làm hai phép đối biểu diễn cho nhanh, và FFT làm được nhờ chọn các điểm là căn của đơn vị, khiến phép đối có cấu trúc chia đôi đệ quy. Họ hàng với tổng tiên tố ở chỗ cả hai đều là “đi đường vòng qua một biểu diễn khác, nơi phép toán cần làm trở thành rẻ”.
- A4. Vì $998\,244\,353 = 119 \cdot 2^{23} + 1$, nên trong số học modulo đó tồn tại căn của đơn vị bậc 2^{23} — điều kiện để chạy biến đổi Fourier trên số nguyên với độ dài là lũy thừa của 2 tới 2^{23} . Modulo $10^9 + 7$ không có tính chất đó, nên các bài cần nhân đa thức lớn buộc phải dùng modulo thân thiện với biến đổi. Nói cách khác, đó là một lựa chọn kỹ thuật có lý do đại số cụ thể, không phải quy ước tùy tiện.

- A5.** Rất khó theo nghĩa thực tiễn: với số hàng trăm chữ số, không thuật toán công khai nào phân tích được trong thời gian hợp lý bằng máy tính cổ điển, dù bài toán này đã được nghiên cứu hàng thế kỷ. Đồng thời, *kiểm tra* một số có nguyên tố hay không lại dễ. Sự bất đối xứng đó cho phép xây một hệ mã mà ai cũng mã hoá được (nhân hai số nguyên tố) nhưng chỉ người giữ khoá mới giải được (biết hai thừa số). Cần lưu ý: đây là độ khó *chưa ai vượt qua*, không phải độ khó đã được chứng minh — chương D/23 sẽ nói vì sao sự phân biệt đó quan trọng.

NÓI LẠI THẬT ĐƠN GIẢN

Số học trong lập trình có một luật chơi khác với toán học ở trường: mọi con số đều bị nhốt trong một cái vòng. Nếu vòng có 12 vạch như mặt đồng hồ thì 10 cộng 5 không ra 15 mà ra 3. Cộng, trừ, nhân đều chạy bình thường trong cái vòng đó. Chỉ có phép chia là rắc rối, vì “chia 2” bây giờ có nghĩa là “nhân với cái gì đó để quay về đúng vị trí ban đầu”, và cái gì đó không phải lúc nào cũng tồn tại. Nó tồn tại khi số bạn chia không có ước chung nào với kích thước vòng — đó là lý do người ta luôn chọn kích thước vòng là số nguyên tố.

Thuật toán cổ nhất còn được dùng tới hôm nay cũng nằm trong chương này, và nó đơn giản tới bất ngờ: muốn tìm ước chung lớn nhất của hai số, cứ lấy số lớn chia số bé rồi thay số lớn bằng phần dư, lặp lại tới khi dư bằng 0. Hai nghìn ba trăm năm và không ai cải tiến được đáng kể.

Còn ý tưởng sâu nhất của chương thì là chuyện đi đường vòng. Nhân hai đa thức dài thì lâu, nhưng nếu thay vì giữ chúng dưới dạng danh sách hệ số, ta ghi lại giá trị của chúng tại một loạt điểm, thì nhân chỉ còn là nhân từng cặp giá trị — rất nhanh. Vấn đề chuyển thành: làm sao đổi qua đổi lại giữa hai cách ghi cho nhanh. Câu trả lời là chọn các điểm thật khéo, và đó là toàn bộ nội dung của phép biến đổi Fourier nhanh.

Cuối cùng là một sự thật đáng nhớ: nhân hai số nguyên tố lớn thì máy tính làm trong tích tắc, nhưng tách ngược một tích ra hai thừa số thì cả thế giới chưa ai làm nổi khi số đủ lớn. Toàn bộ việc mua bán an toàn trên mạng dựa vào đúng sự chênh lệch đó.

Một câu: Trong số học máy tính, mọi thứ chạy trong một cái vòng — và phần lớn lỗi đến từ việc quên mất điều đó.

Chỗ không rút gọn được: Vì sao phân tích thừa số khó thì chưa ai chứng minh được; ta chỉ biết là chưa ai làm được.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Thuật toán Euclid	$\gcd(a, b) = \gcd(b, a \bmod b)$; bất biến cho tính đúng, số thứ hai giảm cho tính dừng; $O(\log \min(a, b))$.
••• Nghịch đảo modulo	Thay cho phép chia; tồn tại khi $\gcd(a, m) = 1$; tính bằng Euclid mở rộng hoặc bằng a^{m-2} khi m nguyên tố.
••• Ba lỗi số học kinh điển	Trần khi nhân hai phần dư cỡ 10^9 ; kết quả âm sau phép trừ rồi lấy dư; chia mà nghịch đảo không tồn tại.
••• Sàng Eratosthenes	$O(n \log \log n)$; biến thể lưu ước nguyên tố nhỏ nhất cho phép phân tích mọi số $\leq n$ trong $O(\log n)$.
••• Bất đối xứng kiểm tra / phân tích	Kiểm nguyên tố thì dễ (Miller–Rabin); phân tích thừa số thì khó — nền của mã hoá khoá công khai.
••• Luỹ thừa nhanh	$a^n = (a^{n/2})^2$; $O(\log n)$. Áp cho ma trận cho các bài truy hồi tuyến tính với n tới 10^{18} .
•• Định lý phần dư Trung Hoa	Phần dư theo các modulo nguyên tố cùng nhau xác định duy nhất số theo tích; dùng để tách phép tính lớn thành nhiều phép tính nhỏ.
•• Ý tưởng của FFT	Đa thức bậc n xác định bởi $n + 1$ giá trị; trong biểu diễn giá trị, nhân là nhân từng điểm $O(n)$; chọn căn của đơn vị để đổi biểu diễn nhanh.
•• Vì sao 998 244 353	Bằng $119 \cdot 2^{23} + 1$ nên có căn của đơn vị bậc 2^{23} — điều kiện chạy biến đổi Fourier trên số nguyên.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
● ● Cơ sở tuyến tính trên $\mathbb{F}_2$	Cộng là XOR; duy trì tập tối thiểu để trả lời “có biểu diễn được không”, “XOR lớn nhất”, “bao nhiêu giá trị phân biệt”.
● ● Mẫu tư duy chung của chương	Đổi biểu diễn để phép toán khó thành dễ: tổng tiền tố, phần dư Trung Hoa, FFT — ba lần cùng một ý.
● Trường hợp xấu của Euclid	Hai số Fibonacci liên tiếp — một liên hệ giữa lý thuyết số và tổ hợp.
● PRIMES is in P	Kiểm tra tính nguyên tố có thuật toán tắt định, đa thức, vô điều kiện (2004); phân tích thừa số thì chưa ai biết. Khe hở giữa hai câu đó là nền của mã hoá khoá công khai. Thực tế vẫn dùng Miller–Rabin.

● ● ● phải thuộc ● ● nên nắm ● biết là có

▮ Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Tính $\binom{n}{k} \bmod p$ cho n tới 10^6 thì làm thế nào?

| Tiên xử lý mảng giai thừa và mảng nghịch đảo giai thừa một lần trong $O(n)$, rồi mỗi truy vấn là một phép nhân ba số trong $O(1)$. Mẹo tính nghịch đảo giai thừa nhanh: tính nghịch đảo của $n!$ một lần bằng lũy thừa, rồi đi ngược $\text{invfact}[i-1] = \text{invfact}[i] \cdot i$. Khi n lớn hơn p thì công thức này không dùng được và cần định lý Lucas.

2. Modulo trong bài của tôi không nguyên tố. Có cách nào tính phân số không?

| Có ba hướng. Nếu số chia nguyên tố cùng nhau với modulo thì nghịch đảo vẫn tồn tại, tính bằng Euclid mở rộng. Nếu không, hãy thử tách modulo thành các lũy thừa nguyên tố, tính theo từng lũy thừa rồi ghép bằng định lý phần dư Trung Hoa. Hoặc, thường là cách tốt nhất, hãy sắp xếp lại công thức để tránh phép chia — ví dụ tính tổ hợp bằng truy hồi Pascal thay vì bằng giai thừa.

3. Làm sao nhân hai số modulo 10^{18} mà không tràn?

| Ba cách. Dùng kiểu số nguyên 128 bit nếu môi trường hỗ trợ — đơn giản nhất. Dùng phép nhân kiểu “nhân bằng cách nhân đôi” theo bit, $O(\log m)$ nhưng chắc chắn. Hoặc dùng mẹo nhân qua số thực dấu phẩy động để ước lượng thương rồi hiệu chỉnh — nhanh nhưng cần cẩn thận về độ chính xác. Trong thi đấu, kiểu 128 bit là lựa chọn mặc định khi có.

4. Khi nào tôi thực sự cần FFT?

| Khi phải tính tích chập của hai dãy dài hơn vài nghìn phần tử — nhân đa thức, đếm số cặp có tổng bằng mỗi giá trị, nhân số lớn. Dưới ngưỡng đó, cách trực tiếp $O(n^2)$ thường nhanh hơn vì hằng số nhỏ hơn nhiều. Dấu hiệu trên đề: bài yêu cầu, với mỗi giá trị s , đếm số cặp (i, j) có $a_i + b_j = s$ — đó chính xác là tích chập.

5. Kiểm tra một số tới 10^{18} có nguyên tố không thì dùng gì?

| Miller–Rabin với một tập nhân chứng cố định đã được kiểm chứng là đủ cho mọi số 64 bit — đây là một trong số ít trường hợp mà thuật toán ngẫu nhiên được “khử ngẫu nhiên” bằng một danh sách nhân chứng cụ thể. Nếu cần phân tích thừa số chứ không chỉ kiểm tra, hãy kết hợp Miller–Rabin với Pollard rho.

6. Bài yêu cầu tính $a^b \bmod m$ với b là số có 10^5 chữ số. Làm sao?

| Dùng định lý Euler: nếu $\gcd(a, m) = 1$ thì $a^{\varphi(m)} \equiv 1$, nên có thể rút gọn số mũ theo modulo $\varphi(m)$. Khi $\gcd(a, m) \neq 1$ thì phải dùng phiên bản mở rộng của định lý, với điều kiện số mũ đủ lớn. Đây là một trong những chỗ dễ sai nhất của chương vì trường hợp biên rất tinh vi — hãy stress test với vết cặn ở m nhỏ.

7. Vì sao chương này lại xếp vào phần “Giới hạn” cùng với NP-khó?

| Vì nó là nơi ta gặp lần đầu một bài toán mà *độ khó là điều đáng mong muốn*. Trong toàn bộ phần A–C, khó là chương ngại. Ở đây, sự khó của bài phân tích thừa số là thứ giữ an toàn cho mọi giao dịch trên mạng. Cách nhìn đó chuẩn bị cho chương D/23, nơi ta học cách *nhận ra* một bài toán là khó và coi đó là thông tin có giá trị chứ không phải thất bại.

Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Euclid và Euclid mở rộng, giải phương trình $ax + by = c$	CSES “Exponentiation”, bài kinh điển[2]	Chú ý điều kiện có nghiệm và cách sinh mọi nghiệm.
● Luỹ thừa nhanh modulo	CSES “Exponentiation”[2]	Chú ý tràn khi nhân và trường hợp số mũ bằng 0.
● Sàng số nguyên tố và đếm ước	CSES “Counting Divisors”[2]	Sàng lưu ước nguyên tố nhỏ nhất; so với cách thử chia từng số.
●● Tính tổ hợp modulo với truy vấn nhiều	CSES “Binomial Coefficients”[2]	Tiền xử lý giai thừa và nghịch đảo giai thừa; mỗi truy vấn $O(1)$.
●● Fibonacci thứ 10^{18} bằng luỹ thừa ma trận	CSES “Fibonacci Numbers”[2]	Kiểm chứng với giá trị nhỏ; chú ý thứ tự nhân ma trận.
●● Phân tích thừa số tới 10^{18}	Bài kinh điển	Miller–Rabin cộng Pollard rho; kiểm bằng cách nhân lại các thừa số.
●● Giải hệ đồng dư bằng định lý phần dư Trung Hoa	Bài kinh điển	Trường hợp modulo không nguyên tố cùng nhau cần xử lý riêng — đừng bỏ qua.
●●● Nhân hai đa thức bằng NTT	Bài kinh điển	Modulo 998 244 353; so thời gian với cách $O(n^2)$ để tìm ngưỡng hoà vốn.
●●● XOR lớn nhất của một tập con	Bài kinh điển	Cơ sở tuyến tính trên $\mathbb{F}_2$; thuật toán tham lam từ bit cao xuống thấp sau khi có cơ sở.
●●● Đếm số cặp có tổng bằng mỗi giá trị	Bài kinh điển	Nhận ra đây là tích chập; đây là bài mẫu để nhận diện dấu hiệu dùng FFT.
●●● Tính $a^b \bmod m$ với b rất lớn	Bài kinh điển	Định lý Euler và phiên bản mở rộng; stress test với m nhỏ vì trường hợp biên rất tinh vi.

Kết chương và đọc tiếp

Chương này là miền mà lỗi hay nằm ở số học chứ không ở ý tưởng, nên ba lỗi ở khối *Cạm bẫy* đáng thuộc hơn bất kỳ công thức nào. Hai điều nữa đáng mang theo: mẫu “đổi biểu diễn để phép toán trở nên dễ”, xuất hiện ba lần trong chương và nhiều lần trong cả quyển sách; và sự bất đối xứng giữa kiểm tra và tìm kiếm, thứ vừa là nền của mật mã vừa là lời mở đầu cho chương D/23.

Chương tiếp theo là miền chuyên thứ hai, và nó có cùng đặc điểm: cạm bẫy không nằm ở ý tưởng mà ở số học. Chỉ khác là ở đó, thủ phạm không phải phép chia dư mà là số dấu phẩy động — và hậu quả nghiêm trọng hơn, vì một phép so sánh sai có thể làm cả thuật toán hình học rơi vào trạng thái không nhất quán (D/22).

Đọc thêm: công trình gốc (chỉ liệt kê nguồn đã qua phản biện; bản tiền ấn được ghi rõ)

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc, đọc phần nào
nền tảng	Agrawal, Kayal, Saxena, "PRIMES Is in P"	Annals of Mathematics, 2004	Mười trang, sơ cấp một cách đáng ngạc nhiên. Đọc phần mở đầu và phát biểu thuật toán, kể cả khi bỏ qua chứng minh.
nền tảng	Cooley & Tukey, "An Algorithm for the Machine Calculation of Complex Fourier Series"	Mathematics of Computation, 1965	Bài gốc của FFT, nền của mục 5. Ngắn, và cho thấy động cơ ban đầu đến từ xử lý tín hiệu.
tra khi cần	Harvey & van der Hoeven, "Integer Multiplication in Time $O(n \log n)$ "	Annals of Mathematics, 2021	Đọc phần mở đầu để nối mục 5 của chương này với biên giới hiện nay của phép nhân ($\mathbb{C}/14$).
tra khi cần	Graham, Knuth, Patashnik, <i>Concrete Mathematics</i>	Addison-Wesley, tái bản 1994	Tra khi cần các tổng và đồng dư xuất hiện trong phân tích; chương về lý thuyết số sơ cấp viết rất kỹ.

Nguồn

- [1] M. Agrawal, N. Kayal, and N. Saxena, “PRIMES Is in P,” *Annals of Mathematics*, vol. 160, no. 2, pp. 781–793, 2004.
- [2] A. Laaksonen. “CSES Problem Set.” (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).
- [3] J. R. Shewchuk, “Adaptive Precision Floating-Point Arithmetic and Fast Robust Geometric Predicates,” *Discrete & Computational Geometry*, vol. 18, pp. 305–363, 1997.
- [4] M. Cygan, F. V. Fomin, Ł. Kowalik, et al., *Parameterized Algorithms*. Springer, 2015.
- [5] R. Impagliazzo and R. Paturi, “On the Complexity of k-SAT,” *Journal of Computer and System Sciences*, vol. 62, no. 2, pp. 367–375, 2001.
- [6] A. Backurs and P. Indyk, “Edit Distance Cannot Be Computed in Strongly Subquadratic Time (Unless SETH Is False),” *SIAM Journal on Computing*, vol. 47, no. 3, 2018, bản hội nghị: STOC 2015.
- [7] A. Gajentaan and M. H. Overmars, “On a Class of $O(n^2)$ Problems in Computational Geometry,” *Computational Geometry: Theory and Applications*, vol. 5, no. 3, pp. 165–185, 1995.
- [8] M. Frigo, C. E. Leiserson, H. Prokop, et al., “Cache-Oblivious Algorithms,” in *Proceedings of the 40th Annual Symposium on Foundations of Computer Science*, 1999, pp. 285–297.
- [9] F. McSherry, M. Isard, and D. G. Murray, “Scalability! But at What COST?” In *15th Workshop on Hot Topics in Operating Systems (HotOS)*, 2015.
- [10] D. D. Sleator and R. E. Tarjan, “Self-Adjusting Binary Search Trees,” *Journal of the ACM*, vol. 32, no. 3, pp. 652–686, 1985.
- [11] N. Alon, Y. Matias, and M. Szegedy, “The Space Complexity of Approximating the Frequency Moments,” *Journal of Computer and System Sciences*, vol. 58, no. 1, pp. 137–147, 1999.
- [12] T. H. Cormen, C. E. Leiserson, R. L. Rivest, et al., *Introduction to Algorithms*, 4th ed. MIT Press, 2022.
- [13] P. Flajolet, É. Fusy, O. Gandouet, et al., “HyperLogLog: The Analysis of a Near-Optimal Cardinality Estimation Algorithm,” in *Conference on Analysis of Algorithms (AofA)*, 2007.

Hình học tính toán (Computational Geometry)

PHỤ THUỘC B/07 cho ngăn xếp trong thuật toán bao lồi và cho hàng đợi ưu tiên trong quét đường; C/14 cho bài cặp điểm gần nhất; D/21 cho tinh thần “cạm bẫy nằm ở số học”.

TÓM TẮT

Hình học tính toán có một đặc điểm khác hẳn mọi chương trước: thuật toán thường đơn giản, nhưng *số dấu phẩy động* có thể phá hủy tính đúng theo những cách rất khó truy. Vì vậy chương bắt đầu từ vị từ định hướng — viên gạch mà gần như mọi thuật toán hình học đứng trên — và từ câu hỏi làm sao tính nó cho đáng tin. Sau đó là bao lồi, quét đường, và các bài kinh điển: giao đoạn thẳng, điểm trong đa giác, cặp điểm gần nhất, cặp điểm xa nhất. Nếu chỉ nhớ một điều: hãy làm việc với số nguyên bất cứ khi nào có thể, và khi buộc phải dùng số thực, hãy hiểu rằng “bằng nhau” không còn là một khái niệm dùng được.

KIẾN THỨC CẦN CÓ TRƯỚC

- **tích có hướng (cross product)** — với hai vector trong mặt phẳng, $u \times v = u_x v_y - u_y v_x$; dấu của nó cho biết v nằm bên trái hay bên phải u .
- **vị từ định hướng** — hàm trả lời “ba điểm này quay trái, quay phải, hay thẳng hàng” — tính bằng dấu của tích có hướng. Nền của gần như mọi thuật toán hình học.
- **bao lồi** — đa giác lồi nhỏ nhất chứa tất cả các điểm đã cho; hình dung là dây thun bọc quanh các đỉnh.
- **quét đường** — kỹ thuật cho một đường thẳng tưởng tượng quét qua mặt phẳng, duy trì một cấu trúc dữ liệu về các đối tượng đang cắt nó.
- **suy biến (degenerate case)** — cấu hình đặc biệt — ba điểm thẳng hàng, hai điểm trùng nhau, đoạn thẳng chồng nhau — nơi phần lớn lỗi hình học nằm.
- **vị từ chính xác** — cách tính vị từ đảm bảo đúng dấu, thường bằng số nguyên hoặc bằng số học nhiều độ chính xác.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao “ba điểm có thẳng hàng không” lại là một câu hỏi khó khi toạ độ là số thực?
- Q2. Bao lồi có thể tính bằng nhiều thuật toán khác nhau. Vì sao cách sắp xếp rồi quét lại phổ biến hơn cả?
- Q3. Kỹ thuật quét đường biến bài toán hai chiều thành bài toán một chiều. Cơ chế nào cho phép làm điều đó?
- Q4. Vì sao một lỗi làm tròn nhỏ trong hình học lại có thể gây ra kết quả hoàn toàn vô nghĩa, chứ không chỉ sai một chút?
- Q5. Khi nào nên dùng số nguyên cho bài hình học, và làm sao tránh tràn khi toạ độ tới 10^9 ?

Có một hiện tượng đặc trưng khi làm bài hình học: chương trình chạy đúng trên mọi ví dụ bạn tự nghĩ ra, rồi sai trên bộ test thật vì có ba điểm thẳng hàng, hoặc hai điểm trùng nhau, hoặc một đoạn thẳng thẳng đứng. Hình học là miền mà *trường hợp suy biến* chiếm phần lớn số lỗi, và điều

đó có lý do sâu xa: thuật toán hình học thường dựa trên các quyết định nhị phân — bên trái hay bên phải, trong hay ngoài — và ở đúng ranh giới giữa hai lựa chọn, mọi thứ trở nên mong manh.

Cộng thêm một tầng khó nữa: nếu toạ độ là số thực, các quyết định đó được tính bằng số dấu phẩy động, vốn có sai số. Một quyết định sai không chỉ làm kết quả lệch một chút mà có thể làm thuật toán rơi vào trạng thái *không nhất quán* — ví dụ tin rằng điểm A ở bên trái điểm B và đồng thời B ở bên trái A — và từ đó cho ra kết quả hoàn toàn vô nghĩa hoặc treo vòng lặp.

Vì vậy chương này ưu tiên hai thứ: các vị từ nền tảng và cách tính chúng cho đáng tin, rồi mới tới thuật toán.

2.1 Vị từ định hướng: viên gạch của mọi thứ

Cho ba điểm A, B, C , câu hỏi cơ bản nhất là: đi từ A tới B rồi tới C , ta rẽ trái, rẽ phải, hay đi thẳng? Câu trả lời nằm ở dấu của tích có hướng

$$(B - A) \times (C - A) = (B_x - A_x)(C_y - A_y) - (B_y - A_y)(C_x - A_x).$$

Dương là rẽ trái, âm là rẽ phải, bằng 0 là thẳng hàng.

Điều đáng nói là gần như mọi thuật toán hình học trong chương này chỉ dùng vị từ đó và một hai vị từ họ hàng. Kiểm tra hai đoạn thẳng có cắt nhau, kiểm tra điểm nằm trong đa giác lồi, xây bao lồi, sắp xếp theo góc — tất cả đều là các cách kết hợp vị từ định hướng. Hệ quả thực hành rất quan trọng: nếu bạn tính vị từ này *đúng dấu* thì phần lớn thuật toán sẽ chạy đúng; nếu tính sai dấu ở các trường hợp gần suy biến thì mọi thứ sụp.

Từ đó có một quy tắc đáng thành phần xạ: *làm việc với số nguyên bất cứ khi nào có thể*. Tích có hướng của các toạ độ nguyên là số nguyên, nên dấu của nó tính được chính xác tuyệt đối — miễn là không tràn. Với toạ độ tới 10^9 , tích có hướng có thể tới $4 \cdot 10^{18}$, sát giới hạn số nguyên 64 bit có dấu; nếu toạ độ lớn hơn hoặc nếu bạn nhân thêm lần nữa thì cần kiểu 128 bit. Đây là câu trả lời cho câu hỏi mở đầu thứ năm.

CẠM BẮY

Khi buộc phải dùng số thực, đừng bao giờ viết `if (a == b)`. Thay bằng so sánh với một ngưỡng: `if (fabs(a-b) < eps)`. Nhưng phải hiểu rằng đây là một sự thoả hiệp chứ không phải lời giải: quan hệ “gần bằng nhau” *không có tính bắc cầu*, nên nếu bạn dùng nó làm quan hệ so sánh trong một hàm sắp xếp, kết quả có thể là không xác định — kể cả sắp chương trình trong một số thư viện. Chọn ε cũng không có công thức chung: quá nhỏ thì không bắt được trường hợp suy biến, quá lớn thì gộp nhầm những điểm thật sự khác nhau.

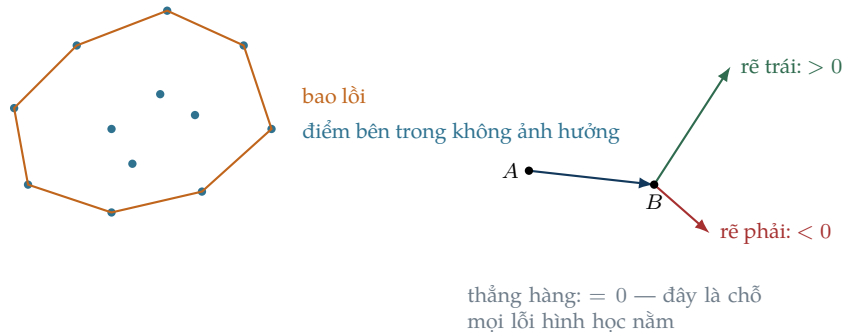
2.2 Bao lồi: thuật toán đầu tiên nên thuộc

Bài toán: cho n điểm, tìm đa giác lồi nhỏ nhất chứa tất cả. Cách hình dung tốt nhất là một sợi dây thun bọc quanh các cây đinh đóng trên bảng.

Cách phổ biến nhất — và cũng dễ cài đúng nhất — là *quét đơn điệu*: sắp các điểm theo hoành độ (rồi tung độ), sau đó xây nửa trên và nửa dưới của bao bằng cách quét một lượt và duy trì một ngăn xếp, loại bỏ mọi điểm làm ta “rẽ sai chiều”. Chi phí $O(n \log n)$, chi phối bởi bước sắp xếp.

Cấu trúc của thuật toán đáng để ý: nó là một *ngăn xếp đơn điệu* (B/07) áp lên tiêu chí hình học thay vì tiêu chí số học. Mỗi điểm vào ngăn xếp một lần và ra tối đa một lần, nên phần quét là $O(n)$ — đúng lập luận khẩu hao quen thuộc.

Bao lồi hữu ích hơn về ngoài của nó vì nhiều bài quy về nó. Cặp điểm *xa nhất* trong tập luôn nằm trên bao lồi, và tìm được bằng kỹ thuật hai con trỏ quay quanh bao. Diện tích nhỏ nhất của hình bao, hình chữ nhật nhỏ nhất bao quanh, kiểm tra một điểm có nằm trong bao — tất cả đều dựa trên bao lồi. Trong tối ưu hoá, bao lồi cũng chính là cấu trúc đẳng sau kỹ thuật tăng tốc quy hoạch động ở C/16: tập các đường thẳng cần lấy cực trị tạo thành một bao lồi.



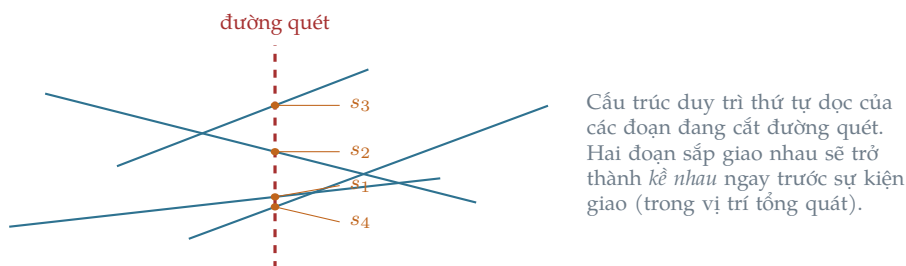
Hình 2.1. Bao lồi và vị từ định hướng. Bên phải là viên gạch: dấu của tích có hướng quyết định mọi thứ. Trường hợp bằng 0 — ba điểm thẳng hàng — là nơi phần lớn lỗi xảy ra, vì đó là ranh giới giữa hai nhánh xử lý, và cũng là nơi sai số dấu phẩy động có thể lật ngược quyết định.

2.3 Quét đường: biến hai chiều thành một chiều

Nhiều bài hình học có dạng “xét mọi cặp đối tượng” và cho $O(n^2)$ một cách hiển nhiên. Kỹ thuật quét đường thường đưa xuống $O(n \log n)$ bằng một ý tưởng chung: cho một đường thẳng đứng quét từ trái sang phải, và duy trì một cấu trúc dữ liệu chứa các đối tượng *đang cắt* đường quét đó, sắp theo tung độ.

Cơ chế làm nó hiệu quả — và đây là câu trả lời cho câu hỏi mở đầu thứ ba — là: tại mỗi thời điểm, ta chỉ cần quan tâm tới quan hệ giữa các đối tượng *kề nhau* trong thứ tự dọc. Hai đoạn thẳng chỉ có thể cắt nhau nếu tại một lúc nào đó chúng trở thành *kề nhau* trên đường quét. Nhờ đó, thay vì xét n^2 cặp, ta chỉ xử lý các sự kiện: một đối tượng bắt đầu, một đối tượng kết thúc, hai đối tượng đổi chỗ. Số sự kiện là $O(n)$ cộng số giao điểm, và mỗi sự kiện tốn $O(\log n)$.

Ba bài kinh điển giải bằng quét đường: tìm mọi giao điểm của n đoạn thẳng; hợp diện tích của n hình chữ nhật; và cặp điểm gần nhất — bài mà chương C/14 giải bằng chia để trị, còn quét đường cho một lời giải khác cũng $O(n \log n)$ nhưng dễ cài hơn với một tập hợp có thứ tự.



Điểm có nằm trong đa giác không. Với đa giác lồi: kiểm bằng vị từ định hướng cho mọi cạnh, hoặc bằng tìm kiếm nhị phân trên góc cho $O(\log n)$. Với đa giác bất kỳ: bắn một tia và đếm số lần cắt biên — lẻ là trong, chẵn là ngoài. Bẫy: tia đi qua đúng một đỉnh sẽ bị đếm hai lần hoặc không lần nào; xử lý điểm trên biên riêng, rồi chỉ xét cạnh có $(y_i > y_p) \neq (y_j > y_p)$ và giao tia về phía phải. Quy ước nửa mở này đếm một đỉnh đúng một lần; yêu cầu hai đầu nghiêm ngặt ở trên/dưới sẽ bỏ sót đỉnh nằm trên tia.

Diện tích đa giác. Công thức dây giày cho diện tích có dấu bằng tổng các tích có hướng chia đôi. Dấu cho biết chiều duyệt, và đó là thông tin hữu ích chứ không phải phiền toái. Bẫy: quên chia 2, và tràn số khi tọa độ lớn.

Hai đoạn thẳng có cắt nhau không. Trường hợp thường thì bốn lần gọi vị từ định hướng là đủ. Bẫy nằm ở trường hợp suy biến: bốn điểm thẳng hàng, khi đó phải kiểm thêm xem hai đoạn có chồng lên nhau không bằng cách so hình chiếu lên hai trục.

Sắp xếp theo góc. Rất nhiều bài cần sắp các điểm quanh một tâm theo góc. Bẫy lớn nhất là dùng hàm arctan — vừa chậm vừa có sai số. Cách đúng là chia các điểm thành nửa trên và nửa dưới rồi so bằng tích có hướng, hoàn toàn bằng số nguyên.

2.5 Vì sao sai số nhỏ lại gây hậu quả lớn

Đây là mục lý giải cho câu hỏi mở đầu thứ tư, và nó là điều phân biệt hình học tính toán với các miền khác.

Trong hầu hết tính toán số, một sai số nhỏ cho một kết quả sai nhỏ — bạn tính ra 3,0001 thay vì 3,0000 và không sao. Trong hình học, các đại lượng số được dùng để đưa ra *quyết định rời rạc*: điểm này nằm trong hay ngoài, ba điểm này thẳng hàng hay không. Một sai số nhỏ ở gần ranh giới làm lật ngược quyết định, và từ một quyết định sai, cấu trúc dữ liệu của thuật toán rơi vào trạng thái không tương ứng với bất kỳ cấu hình hình học hợp lệ nào.

Hậu quả không phải là “kết quả lệch một chút” mà là các sự cố có tính chất khác hẳn: bao lỗi tự cắt chính nó, thuật toán quét đường lặp vô hạn vì thứ tự trên đường quét mâu thuẫn, hoặc chương trình sập vì một hàm sắp xếp nhận được quan hệ so sánh không nhất quán.

Có ba cách đối phó, xếp theo thứ tự nên thử. *Một*: dùng số nguyên — luôn ưu tiên, và trong lập trình thi đấu thì đề bài thường cho tọa độ nguyên chính là vì lý do này. *Hai*: dùng số học chính xác cho riêng các vị từ, tức là chỉ trả giá ở những chỗ quyết định. *Ba*: nếu chấp nhận dung sai, phải định nghĩa rõ bài toán xấp xỉ và quy tắc xử lý suy biến. Phép quay bảo toàn tính thẳng hàng nên không thể loại bỏ ba điểm thẳng hàng; nhiều tọa độ thì thay đổi dữ liệu và có thể thay đổi đáp án. Khi cần dấu chính xác, dùng vị từ thích nghi với kiểm soát sai số[3].

2.6 Vị từ thích nghi: trả giá chính xác đúng chỗ cần

Cách thứ hai ở mục trên — “dùng số học chính xác cho riêng các vị từ” — nghe như một lời khuyên chung chung, nhưng nó có một cài đặt cụ thể đã trở thành chuẩn công nghiệp, và cơ chế của nó đáng hiểu vì nó là một mẫu thiết kế dùng lại được.

Vấn đề với số học chính xác thuần túy là giá: nếu mọi phép định hướng đều tính bằng số hữu tỉ chính xác, chương trình chậm đi hàng chục lần, trong khi hầu hết các lần gọi đều ở xa ranh giới tới mức số thực thường đã cho dấu đúng. Ý tưởng của Shewchuk[3] là *thích nghi*: tính vị từ theo nhiều tầng, tầng đầu rẻ nhất, và mỗi tầng đi kèm một *chặn sai số* tính được ngay trong lúc chạy.

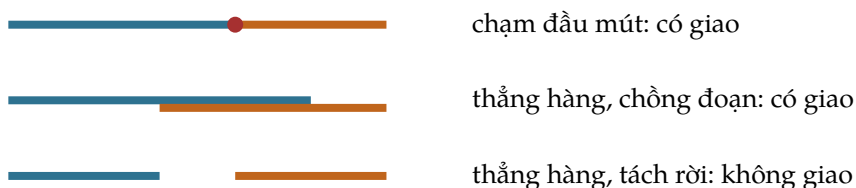
Nếu giá trị ước lượng lớn hơn chặn sai số của nó thì dấu chắc chắn đúng và ta dừng — đây là trường hợp của gần như mọi dữ liệu thật. Chỉ khi giá trị rơi vào vùng không phân biệt được với 0, thuật toán mới leo lên tầng chính xác hơn, và chỉ ở những cấu hình gần suy biến ta mới trả giá đầy đủ.

Kết quả là một hàm vừa *luôn* cho dấu đúng vừa nhanh gần bằng số thực trên dữ liệu bình thường — và đó là lý do các thư viện tam giác phân, lưới và bản đồ đều dựa trên loại vị từ này. Mẫu tư duy chung, đáng mang sang chỗ khác: *tính rẻ trước kèm một chứng chỉ về sai số, và chỉ nâng cấp chi phí khi chứng chỉ không đủ để kết luận*. Cùng khuôn ấy xuất hiện ở bộ lọc trước một phép kiểm đất tiền (B/08) và ở việc cắt nhánh trong quay lui (C/13).

Một hệ quả thực hành đi kèm, thường bị bỏ qua: nếu bạn so sánh bằng ngưỡng ε , quan hệ “bằng nhau” sinh ra *không bắc cầu* — có thể $a \approx b$ và $b \approx c$ mà $a \not\approx c$. Đưa một quan hệ so sánh như vậy cho hàm sắp xếp của thư viện là hành vi không xác định, và triệu chứng của nó là chương trình sập ở một chỗ chẳng liên quan gì tới hình học. Đây chính là loại sự cố mà mục trước gọi là “hậu quả khác hẳn”, và cách phòng duy nhất chắc chắn là đừng để ε lọt vào quan hệ thứ tự.

2.7 Bộ hình suy biến phải thử trước dữ liệu ngẫu nhiên

Hình 2.3 gom ba cấu hình mà phép kiểm chỉ dựa vào dấu nghiêm ngặt thường bỏ sót. Trước khi viết code, quyết định “giao” có tính chạm đầu mút không; hình dưới dùng đoạn đóng, nên hai trường hợp đầu đều có giao.



Hình 2.3. Ba ca kiểm thử cho giao hai đoạn đóng. Hàng giữa dịch nét màu cam xuống một chút để nhìn thấy phần chồng; dữ liệu toán học vẫn có cùng tung độ. Ba cấu hình đều có định hướng bằng 0, nên phải kiểm thêm hình chiếu hoặc điểm nằm trên đoạn.

Kinh nghiệm số học: ép sang kiểu rộng *trước* phép trừ và nhân, không đợi tới khi gán kết quả. Chẳng hạn `wide(B.x) - wide(A.x)` bảo vệ phép trừ; ép kiểu sau `B.x - A.x` không cứu được phép trừ đã tràn. Với dấu của hai định hướng, so dấu trực tiếp thay vì nhân chúng, vì tích hai định thức có thể tràn dù từng định thức còn vừa kiểu số. Thêm các ca điểm trùng nhau, đoạn dài 0 và tọa độ ở hai cực của miền cho phép.

2.8 Gốc rễ: một luận án năm 1978 và cuộc khủng hoảng về độ tin cậy

Hình học tính toán là ngành trẻ so với các chương khác: nó thường được tính là bắt đầu từ luận án tiến sĩ của Michael Shamos năm 1978, quyển đầu tiên coi “các bài toán hình học” như một lĩnh vực có phương pháp riêng thay vì một tập hợp bài lẻ. Trước đó, các thuật toán hình học tồn tại rải rác — thuật toán bao lồi của Graham ra đời năm 1972, và động cơ của nó rất thực tế: xử lý dữ liệu thống kê, nơi bao lồi được dùng để tìm các điểm cực biên.

Kỹ thuật quét đường được hệ thống hoá cuối thập niên 1970, với thuật toán tìm giao điểm các đoạn thẳng của Bentley và Ottmann năm 1979 — cùng Bentley của *Programming Pearls* ở chương

A/02. Cùng thời kỳ, các cấu trúc chia không gian như cây k -d và cây khoảng ra đời để trả lời truy vấn hình học, và chúng chính là các cây tìm kiếm được tăng cường mà B/09 đã nói.

Nhưng câu chuyện đáng nhớ nhất của ngành này là câu chuyện về *độ tin cậy số học*, và nó đáng kể vì nó là một trường hợp hiếm hoi mà cả một cộng đồng phải thừa nhận rằng lý thuyết đã bỏ sót một điều căn bản. Suốt thập niên 1980, các thuật toán hình học được công bố với chứng minh đầy đủ, nhưng khi cài đặt bằng số thực thì chúng sập hoặc cho kết quả vô nghĩa trên dữ liệu thật. Vấn đề không phải lỗi lập trình mà là sự lệch pha giữa mô hình — “số thực chính xác vô hạn” — và thực tế — “số dấu phẩy động 64 bit”.

Phản ứng của ngành trong thập niên 1990 là xây dựng *tính toán vị từ chính xác*: các kỹ thuật tính dấu của biểu thức hình học một cách đảm bảo, thường bằng cách ước lượng trước với số dấu phẩy động rồi chỉ chuyển sang số học chính xác khi kết quả gần 0. Ý tưởng “chỉ trả giá cho độ chính xác ở nơi thật sự cần” này về sau thành chuẩn, và nó được đóng gói trong các thư viện hình học công nghiệp. Bài học cho người học: khi ai đó nói “thuật toán này đã được chứng minh”, hãy hỏi thêm “chứng minh trong mô hình số nào”.

Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
1972 — Graham	Tìm các điểm cực biên trong dữ liệu thống kê.	Thuật toán bao lồi bằng sắp xếp theo góc và quét bằng ngăn xếp.
1978 — Shamos	Các bài toán hình học được giải lẻ tẻ, không có phương pháp chung.	Luận án khai sinh hình học tính toán như một lĩnh vực với các kỹ thuật chuẩn.
1979 — Bentley & Ottmann	Tìm mọi giao điểm của n đoạn thẳng nhanh hơn $O(n^2)$.	Quét đường: biến bài hai chiều thành chuỗi sự kiện một chiều.
Thập niên 1980 — cấu trúc truy vấn hình học	Trả lời nhiều truy vấn trên cùng tập điểm.	Cây k -d, cây khoảng, cây phân đoạn — đều là cây tìm kiếm được tăng cường (B/09, B/10).
Thập niên 1990 — khủng hoảng độ tin cậy	Thuật toán đã chứng minh vẫn sập khi cài bằng số thực.	Nhận ra khoảng cách giữa mô hình số thực lý tưởng và số dấu phẩy động; ra đời kỹ thuật vị từ chính xác thích ứng.
Thập niên 1990–2000 — thư viện hình học	Mỗi nhóm tự cài lại và tự mắc lại cùng các lỗi.	Các thư viện đóng gói vị từ chính xác và xử lý suy biến, cho phép dùng hình học tính toán trong sản phẩm thật.

2.9 Liên hệ ngang

Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Đồ hoạ máy tính	Cắt đa giác, kiểm tra va chạm, dò tia	Cùng vị từ nền tảng. Khác về ưu tiên: đồ hoạ chấp nhận sai số nhỏ để đổi lấy tốc độ khung hình, còn hình học tính toán thường cần tính đúng tuyệt đối — hai cộng đồng cùng thuật toán, khác tiêu chuẩn.
Hệ thông tin địa lý	Chồng bản đồ, truy vấn vùng, tìm điểm gần nhất	Quét đường và cây phân vùng không gian là công cụ nền. Ở đây dữ liệu suy biến rất phổ biến (đường biên chung, điểm trùng), nên xử lý suy biến quan trọng hơn cả tốc độ.

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu (tiếp)
Robot và SLAM	Kiểm tra va chạm, bản đồ không gian tự do, khớp đám mây điểm	Bao lồi dùng để mô tả vùng chiếm chỗ của vật thể; cây k -d dùng để tìm điểm gần nhất trong bước khớp đám mây. Điểm khác: dữ liệu cảm biến có nhiều nên các vị từ chính xác ít giá trị hơn — cái cần là bền vững với nhiễu, không phải chính xác tuyệt đối.
Thiết kế và chế tạo	CAD, sinh lưới, in ba chiều	Sinh lưới tam giác dựa trên tam giác phân Delaunay; các lỗi suy biến ở đây gây hậu quả vật lý thật, nên ngành này là nơi vị từ chính xác được coi trọng nhất.
Tối ưu hoá	Bao lồi trong quy hoạch tuyến tính	Miền khả thi của quy hoạch tuyến tính là một đa diện lồi, và thuật toán đơn hình đi trên các đỉnh của nó. Bao lồi trong chương này là bản hai chiều của cùng đối tượng.
Quy hoạch động	Kỹ thuật bao lồi để tăng tốc chuyển trạng thái	Tập các đường thẳng cần lấy cực trị tạo thành một bao lồi; kỹ thuật ở C/16 chính là bao lồi động. Một liên hệ đẹp giữa hình học và tối ưu tổ hợp.
Học máy	Máy vector hỗ trợ, phân tách tuyến tính	Việc tìm siêu phẳng phân tách hai lớp có liên hệ trực tiếp với khoảng cách giữa hai bao lồi của hai lớp — hình học lồi là ngôn ngữ chung.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì với số thực, “bằng 0” gần như không bao giờ xảy ra đúng nghĩa: kết quả tính tích có hướng của ba điểm thẳng hàng có thể ra 10^{-16} thay vì 0 do sai số làm tròn. Bạn buộc phải so với một ngưỡng, mà ngưỡng đó không có giá trị đúng phổ quát — nó phụ thuộc độ lớn của tọa độ và số phép toán đã thực hiện. Tệ hơn, quan hệ “gần bằng” không bắc cầu, nên nó không dùng được ở những nơi cần một quan hệ so sánh nhất quán.
- A2. Vì nó dễ cài đúng và ít trường hợp riêng nhất. Sau khi sắp xếp, thuật toán chỉ là một vòng quét với ngăn xếp và một phép kiểm tra vị từ định hướng — ít chỗ để sai. Các thuật toán khác như gói quà có thể nhanh hơn khi bao có ít đỉnh, nhưng chúng nhạy cảm hơn với các điểm thẳng hàng và trùng nhau. Trong hình học, tiêu chí “ít trường hợp riêng” đáng giá hơn một chút khác biệt về tiệm cận.
- A3. Cơ chế là: tại mỗi thời điểm, chỉ các đối tượng *kề nhau* theo thứ tự dọc trên đường quét mới có thể tương tác. Hai đoạn thẳng chỉ cắt nhau nếu có lúc chúng *kề nhau* trong thứ tự đó. Nhờ vậy, thay vì n^2 cặp, ta chỉ cần xử lý các sự kiện làm thay đổi quan hệ *kề*: một đối tượng vào, một đối tượng ra, hai đối tượng đổi chỗ. Bài toán hai chiều trở thành bài toán duy trì một tập có thứ tự trong một chiều, tức là quay về cấu trúc dữ liệu của B/09.
- A4. Vì các đại lượng số ở đây được dùng để ra *quyết định rời rạc*, không phải để báo cáo giá trị. Sai số nhỏ ở gần ranh giới lật ngược quyết định, và một quyết định sai đưa cấu trúc dữ liệu vào trạng thái không tương ứng với cấu hình hình học hợp lệ nào — ví dụ thứ tự trên đường quét mâu thuẫn với chính nó. Từ đó hậu quả không phải sai số nhỏ mà là bao lồi tự cắt, vòng lặp không dừng, hoặc chương trình sập.
- A5. Luôn nên dùng số nguyên khi đề cho tọa độ nguyên — và đề thi thường cố ý cho như vậy chính vì lý do này. Với tọa độ tới 10^9 , tích có hướng có thể tới $4 \cdot 10^{18}$, vẫn nằm trong số nguyên 64 bit có dấu nhưng rất sát; nếu công thức của bạn nhân thêm một lần nữa hoặc cộng nhiều số hạng thì phải chuyển sang kiểu 128 bit. Mẹo giảm rủi ro: tính tiến toàn bộ tập điểm về gần gốc tọa độ trước khi tính, vì tích có hướng chỉ phụ thuộc hiệu các tọa độ.

NÓI LẠI THẬT ĐƠN GIẢN

Hình học trên máy tính có một điều lạ: bài toán thì trực quan, nhưng cái làm hỏng chương trình lại không nằm ở chỗ bạn nghĩ.

Gần như mọi thuật toán trong chương này đứng trên một câu hỏi duy nhất: đi từ điểm A tới B rồi tới C thì bạn rẽ trái hay rẽ phải? Có một công thức nhỏ trả lời câu đó bằng một phép nhân và một

phép trừ. Bao lỗi, kiểm tra hai đoạn cắt nhau, kiểm tra điểm nằm trong hình — tất cả chỉ là cách xâu chuỗi câu hỏi ấy lại.

Rắc rối bắt đầu khi ba điểm nằm gần như thẳng hàng. Lúc đó câu trả lời đúng là “không rẽ bên nào cả”, nhưng máy tính dùng số thực có sai số, nên nó có thể nói “rẽ trái một chút” hoặc “rẽ phải một chút” tùy vào bụi bặm của phép làm tròn. Và vì thuật toán dựa trên câu trả lời đó để quyết định, một câu trả lời sai không làm kết quả lệch chút xíu — nó làm chương trình tin vào một thế giới không tồn tại, kiểu như tin rằng A ở bên trái B và B cũng ở bên trái A. Từ đó thì bất cứ chuyện gì cũng có thể xảy ra: hình vẽ tự cắt chính nó, vòng lặp chạy mãi, hoặc chương trình sập.

Cách phòng đơn giản nhất và hiệu quả nhất: nếu toạ độ là số nguyên thì hãy giữ nguyên số nguyên từ đầu tới cuối, đừng đổi sang số thực. Phép nhân và trừ trên số nguyên cho kết quả chính xác tuyệt đối, nên câu hỏi “rẽ bên nào” luôn được trả lời đúng. Chỉ cần canh chừng một chuyện: số nguyên cũng có giới hạn, và với toạ độ cỡ một tỉ thì tích của chúng đã sát trần.

Một câu: Mọi thuật toán hình học đứng trên một câu hỏi “rẽ trái hay rẽ phải” — và tính đúng của cả chương trình phụ thuộc vào việc trả lời câu đó cho chính xác.

Chỗ không rút gọn được: Không có giá trị ε đúng phổ quát; việc chọn nó luôn là thỏa hiệp phụ thuộc dữ liệu cụ thể.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Vị từ định hướng	Dấu của $(B - A) \times (C - A)$: dương là rẽ trái, âm là rẽ phải, 0 là thẳng hàng. Nền của gần như mọi thuật toán hình học.
••• Quy tắc số học	Dùng số nguyên bất cứ khi nào có thể; toạ độ tới 10^9 cho tích có hướng tới $4 \cdot 10^{18}$ — sát trần 64 bit.
••• Vì sao sai số nhỏ gây hậu quả lớn	Số được dùng để ra quyết định rồi rắc; quyết định sai đưa cấu trúc vào trạng thái không tương ứng cấu hình hợp lệ nào $\Rightarrow$ tự cắt, lặp vô hạn, sập.
••• Bao lỗi bằng quét đơn điệu	Sắp theo hoành độ rồi quét hai nửa với ngăn xếp đơn điệu; $O(n \log n)$, chi phối bởi sắp xếp; ít trường hợp riêng nhất.
••• Cơ chế của quét đường	Chỉ các đối tượng <i>kê nhau</i> theo thứ tự dọc mới tương tác; xử lý sự kiện vào/ra/đổi chỗ thay vì mọi cặp.
•• So sánh số thực	Không bao giờ dùng $==$; dùng ngưỡng ε nhưng nhớ rằng “gần bằng” <i>không bắc cầu</i> nên không dùng làm quan hệ sắp xếp.
•• Điểm trong đa giác	Lỗi: kiểm vị từ mọi cạnh, hoặc nhị phân theo góc $O(\log n)$. Bất kỳ: bắn tia đếm số lần cắt, lẻ là trong; bẫy là tia đi qua đỉnh.
•• Diện tích đa giác	Công thức dây giày: tổng tích có hướng chia đôi; dấu cho biết chiều duyệt.
•• Sắp xếp theo góc	Chia nửa trên / nửa dưới rồi so bằng tích có hướng — hoàn toàn số nguyên; đừng dùng arctan.
•• Cặp điểm xa nhất	Luôn nằm trên bao lỗi; tìm bằng hai con trỏ quay quanh bao sau khi có bao.
•• Ba cách đối phó sai số	Số nguyên (ưu tiên); vị từ chính xác thích ứng (chỉ trả giá ở chỗ quyết định); hoặc định nghĩa dung sai và xử lý suy biến rõ ràng. Phép quay không loại tính thẳng hàng.
• Liên hệ với quy hoạch động	Kỹ thuật bao lỗi tăng tốc chuyển trạng thái ($C/16$) chính là bao lỗi động trên tập đường thẳng.
•• Vị từ thích nghi	Tính rẽ trước kèm một chặn sai số tính ngay lúc chạy; chỉ leo lên tầng chính xác hơn khi giá trị không phân biệt được với 0. Kèm theo: so sánh bằng ε không bắc cầu, đừng đưa vào hàm sắp xếp.

••• phải thuộc •• nên nắm • biết là có

Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Tôi nên lưu điểm bằng kiểu gì trong bài hình học?

| Nếu đề cho toạ độ nguyên, hãy dùng số nguyên 64 bit và giữ nguyên suốt — kể cả khi công thức trung gian trông có vẻ cần chia. Rất nhiều bài tránh được phép chia bằng cách nhân chéo thay vì so hai phân số. Chỉ chuyển sang số thực ở bước cuối cùng khi phải in ra kết quả có phần thập phân. Nếu đề cho toạ độ thực, hãy cân nhắc nhân tất cả lên một hệ số rồi làm tròn — nhiều bài cho toạ độ với số chữ số thập phân cố định chính là gợi ý làm điều đó.

2. Bao lỗi của tôi sai khi có nhiều điểm thẳng hàng. Xử lý thế nào?

| Vấn đề nằm ở chỗ bạn dùng dấu “lớn hơn” hay “lớn hơn hoặc bằng” khi loại điểm khỏi ngăn xếp, và câu trả lời phụ thuộc vào việc đề muốn gì: giữ mọi điểm trên cạnh của bao, hay chỉ giữ các đỉnh thật sự. Hãy đọc kỹ đề, chọn một quy ước, và kiểm bằng một ví dụ có ba điểm thẳng hàng trên biên. Ngoài ra nhớ loại các điểm trùng nhau trước khi chạy, vì chúng gây ra tích có hướng bằng 0 ở những chỗ không mong muốn.

3. Có nên tự cài các thuật toán hình học phức tạp trong thi đấu không?

| Với bao lỗi, giao đoạn thẳng, diện tích, sắp xếp theo góc thì có — chúng ngắn và bạn nên có sẵn trong thư viện riêng. Với tam giác phân Delaunay, giao nửa mặt phẳng, hay bao lỗi ba chiều thì hãy cân nhắc: chúng dài, nhiều trường hợp suy biến, và thường có cách vòng qua. Trước khi cài một thuật toán hình học nặng, hãy dành năm phút hỏi “bài này có cách nào không cần hình học không” — nhiều bài trông hình học lại giải được bằng sắp xếp cộng một quan sát.

4. Làm sao kiểm thử một chương trình hình học?

| Ba lớp dữ liệu, và lớp thứ ba là quan trọng nhất. Một: dữ liệu ngẫu nhiên toạ độ lớn — bắt lỗi tràn. Hai: dữ liệu ngẫu nhiên toạ độ *nhỏ*, ví dụ trong $[-3, 3]$ — điều này tạo ra rất nhiều điểm thẳng hàng và trùng nhau, tức là ép trường hợp suy biến xuất hiện. Ba: dữ liệu dựng tay cho từng trường hợp suy biến bạn nghĩ ra được. Lớp thứ hai là mẹo hiệu quả nhất và bị bỏ qua nhiều nhất — toạ độ nhỏ khiến suy biến xảy ra liên tục thay vì hiếm.

5. Vì sao đề bài thi thường cho toạ độ nguyên?

| Chính vì lý do ở mục 6: với toạ độ nguyên, mọi vị từ tính được chính xác tuyệt đối, nên bài toán có một đáp án xác định và việc chấm không phụ thuộc vào cách cài số thực của thí sinh. Nếu bạn thấy đề cho toạ độ thực với số chữ số thập phân giới hạn, gần như chắc chắn ý định là bạn nhân lên thành số nguyên. Đây là một gợi ý đọc đề mà nhiều người bỏ qua.

6. Cây k -d và bao lỗi khác nhau về vai trò thế nào?

| Bao lỗi là một *kết quả* — nó mô tả hình dạng bao ngoài của tập điểm. Cây k -d là một *cấu trúc tra cứu* — nó tổ chức tập điểm để trả lời nhiều truy vấn về lân cận. Chúng trả lời hai loại câu hỏi khác nhau: “hình bao của tập này là gì” so với “điểm nào gần điểm truy vấn nhất”. Trong ứng dụng robot, bao lỗi hay dùng để mô tả vật cản, còn cây k -d dùng để khớp đám mây điểm.

7. Xoay hệ toạ độ một góc ngẫu nhiên có thực sự giúp gì không?

| Có, và nó là một mẹo thực dụng đáng biết. Rất nhiều trường hợp suy biến trong quét đường liên quan tới các đoạn thẳng đứng hoặc các điểm cùng hoành độ. Xoay toàn bộ tập điểm một góc ngẫu nhiên nhỏ làm xác suất còn tồn tại các trường hợp đó gần bằng 0, nên bạn không phải viết mã xử lý riêng cho chúng. Cái giá: bạn buộc phải chuyển sang số thực, nên đây là đánh đổi giữa hai loại rủi ro — và nó chỉ đáng khi mã xử lý suy biến phức tạp hơn nhiều so với rủi ro sai số.

Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Vị từ định hướng và kiểm tra hai đoạn thẳng cắt nhau	CSES “Line Segment Intersection”[2]	Xử lý đủ trường hợp thẳng hàng chồng nhau; kiểm bằng toạ độ nhỏ.
● Diện tích đa giác bằng công thức dây giày	CSES “Polygon Area”[2]	Giữ nguyên số nguyên, in ra diện tích nhân đôi để tránh số thực.

Bài	Nguồn	Điều cần nhận ra (tiếp)
• Điểm nằm trong, trên biên hay ngoài đa giác	CSES “Point in Polygon”[2]	Bắn tia; xử lý trường hợp điểm nằm đúng trên biên trước.
•• Bao lồi	CSES “Convex Hull”[2]	Quét đơn điệu; quyết định rõ quy ước về điểm thẳng hàng trên biên.
•• Cặp điểm gần nhất	CSES “Minimum Euclidean Distance”[2]	Chia để trị ($C/14$) hoặc quét đường với tập có thứ tự; cài cả hai nếu có thời gian.
•• Số điểm nguyên trên biên và bên trong đa giác	CSES “Polygon Lattice Points”[2]	Định lý Pick cộng gcd cho số điểm trên cạnh — một liên hệ đẹp với $D/21$.
••• Tìm mọi giao điểm của n đoạn thẳng	Bài kinh điển	Quét đường với tập có thứ tự; đây là bài kiểm tra khả năng xử lý suy biến của bạn.
••• Hợp diện tích của n hình chữ nhật	Bài kinh điển	Quét đường cộng cây phân đoạn ($B/10$); một trong những bài ghép hai chương đẹp nhất.
••• Cặp điểm xa nhất trong tập	Bài kinh điển	Bao lồi rồi hai con trỏ quay quanh bao; chứng minh vì sao đáp án luôn nằm trên bao.
••• Hình chữ nhật nhỏ nhất bao quanh một tập điểm	Bài kinh điển	Một cạnh của hình chữ nhật tối ưu luôn trùng một cạnh của bao lồi — hãy tự chứng minh trước.
••• Kiểm thử bao lồi bằng dữ liệu toạ độ nhỏ	Bài tập kiểm thử	Sinh ngẫu nhiên toạ độ trong $[-3, 3]$ và so với vết cạn; mẹo hiệu quả nhất của chương.

Kết chương và đọc tiếp

Hình học là miền mà thuật toán đơn giản còn cài đặt thì không, và lý do nằm ở chỗ số học được dùng để ra quyết định rời rạc. Ba điều đáng mang theo: vị từ định hướng là viên gạch của mọi thứ; số nguyên là bạn còn số thực là nguồn rủi ro; và mẹo kiểm thử bằng toạ độ nhỏ để ép trường hợp suy biến xuất hiện.

Hai chương đầu của phần D là hai miền chuyên. Chương tiếp theo là chương quan trọng nhất của cả phần, và nó thay đổi cách bạn phân bổ thời gian khi gặp bài lạ: cách nhận ra một bài toán *không* có lời giải nhanh nào được biết tới, vì sao thông tin đó đáng giá, và bốn cách sống chung với nó ($D/23$).

Đọc thêm: công trình gốc (chỉ liệt kê nguồn đã qua phản biện; bản tiền ấn được ghi rõ)

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc, đọc phần nào
nền tảng	Shewchuk, “Adaptive Precision Floating-Point Arithmetic and Fast Robust Geometric Predicates”	Discrete & Computational Geometry, 1997	Nguồn của vị từ thích nghi. Đọc phần mở đầu và phần chặn sai số; mã nguồn kèm theo dùng được trực tiếp.
nền tảng	de Berg, Cheong, van Kreveld, Overmars, <i>Computational Geometry: Algorithms and Applications</i>	Springer, tái bản 2008	Sách trực chính của chương. Đọc chương bao lồi và chương quét đường; phần suy biến nằm rải rác trong các mục “degeneracies”.
tra khi cần	Gajentaan & Overmars, “On a Class of $O(n^2)$ Problems in Computational Geometry”	Computational Geometry: Theory and Applications, 1995	Vì sao nhiều bài hình học phẳng đứng yên ở n^2 : chúng khó ngang bài 3SUM ($D/24$).
tra khi cần	Knuth, <i>The Art of Computer Programming</i> , tập 1	Addison-Wesley, tái bản 1997	Tra khi cần cơ sở về số học máy tính và biểu diễn dấu phẩy động.

Nguồn

- [1] M. Agrawal, N. Kayal, and N. Saxena, “PRIMES Is in P,” *Annals of Mathematics*, vol. 160, no. 2, pp. 781–793, 2004.
- [2] A. Laaksonen. “CSES Problem Set.” (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).
- [3] J. R. Shewchuk, “Adaptive Precision Floating-Point Arithmetic and Fast Robust Geometric Predicates,” *Discrete & Computational Geometry*, vol. 18, pp. 305–363, 1997.
- [4] M. Cygan, F. V. Fomin, Ł. Kowalik, et al., *Parameterized Algorithms*. Springer, 2015.
- [5] R. Impagliazzo and R. Paturi, “On the Complexity of k-SAT,” *Journal of Computer and System Sciences*, vol. 62, no. 2, pp. 367–375, 2001.
- [6] A. Backurs and P. Indyk, “Edit Distance Cannot Be Computed in Strongly Subquadratic Time (Unless SETH Is False),” *SIAM Journal on Computing*, vol. 47, no. 3, 2018, bản hội nghị: STOC 2015.
- [7] A. Gajentaan and M. H. Overmars, “On a Class of $O(n^2)$ Problems in Computational Geometry,” *Computational Geometry: Theory and Applications*, vol. 5, no. 3, pp. 165–185, 1995.
- [8] M. Frigo, C. E. Leiserson, H. Prokop, et al., “Cache-Oblivious Algorithms,” in *Proceedings of the 40th Annual Symposium on Foundations of Computer Science*, 1999, pp. 285–297.
- [9] F. McSherry, M. Isard, and D. G. Murray, “Scalability! But at What COST?” In *15th Workshop on Hot Topics in Operating Systems (HotOS)*, 2015.
- [10] D. D. Sleator and R. E. Tarjan, “Self-Adjusting Binary Search Trees,” *Journal of the ACM*, vol. 32, no. 3, pp. 652–686, 1985.
- [11] N. Alon, Y. Matias, and M. Szegedy, “The Space Complexity of Approximating the Frequency Moments,” *Journal of Computer and System Sciences*, vol. 58, no. 1, pp. 137–147, 1999.
- [12] T. H. Cormen, C. E. Leiserson, R. L. Rivest, et al., *Introduction to Algorithms*, 4th ed. MIT Press, 2022.
- [13] P. Flajolet, É. Fusy, O. Gandouet, et al., “HyperLogLog: The Analysis of a Near-Optimal Cardinality Estimation Algorithm,” in *Conference on Analysis of Algorithms (AofA)*, 2007.

Độ khó và quy dẫn (Hardness and Reductions)

PHỤ THUỘC C/13 cho vết cạo có cắt tia — công cụ chính khi gặp bài khó; C/15 cho xấp xỉ tham lam; C/16 cho khái niệm giả đa thức.

TÓM TẮT

Chương này thay đổi cách bạn phân bổ thời gian khi gặp bài lạ. Nó định nghĩa khái niệm P và NP theo hướng thực dụng — điểm mấu chốt là sự khác biệt giữa *tìm* lời giải và *kiểm* một lời giải cho sẵn — rồi dạy kỹ năng nhận ra một bài toán là NP-khó bằng cách quy dẫn từ một bài trong danh mục. Nửa sau là phần quan trọng hơn với người làm thực tế: bốn cách sống chung với độ khó, gồm thuật toán mũ thông minh, xấp xỉ có bảo đảm, tham số hoá, và bộ giải công nghiệp. Nếu chỉ nhớ một điều: biết một bài là NP-khó không phải thất bại mà là thông tin đắt giá nhất bạn có thể có — nó ngăn bạn mất ba ngày tìm một lời giải đa thức không tồn tại, và chuyển câu hỏi từ “giải thế nào” sang “chấp nhận đánh đổi nào”.

KIẾN THỨC CẦN CÓ TRƯỚC

- **bài toán quyết định** — bài toán có câu trả lời đúng/sai, ví dụ “đồ thị này có tập độc lập kích thước k không”. Lý thuyết độ phức tạp phát biểu qua dạng này cho gọn.
- **lớp P** — các bài toán quyết định giải được trong thời gian đa thức theo kích thước dữ liệu vào.
- **lớp NP** — các bài toán mà một lời giải, nếu được cho sẵn, *kiểm chứng* được trong thời gian đa thức. Chú ý: NP không có nghĩa là “không đa thức”.
- **quy dẫn** — biến đổi bài toán A thành bài toán B trong thời gian đa thức, sao cho lời giải của B cho ngay lời giải của A. Ý nghĩa: B khó ít nhất bằng A.
- **NP-đầy đủ** — thuộc NP và mọi bài trong NP đều quy dẫn về được; đây là các bài “khó nhất trong NP”.
- **NP-khó** — khó ít nhất bằng mọi bài trong NP, nhưng không nhất thiết thuộc NP — ví dụ các bài tối ưu tương ứng.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao lý thuyết lại phát biểu mọi thứ qua bài toán quyết định, trong khi thực tế ta luôn muốn bài toán tối ưu?
- Q2. Bài ba lô có lời giải quy hoạch động $O(nW)$. Vì sao điều đó không mâu thuẫn với việc nó là NP-khó?
- Q3. Khi chứng minh bài của bạn là NP-khó, quy dẫn phải đi theo chiều nào — và vì sao chiều ngược lại là một lỗi vô nghĩa?
- Q4. Nếu một bài là NP-khó, vì sao trong công nghiệp người ta vẫn giải được những trường hợp hàng triệu biến?
- Q5. Biết một bài là NP-khó thì bạn thay đổi cụ thể điều gì trong cách làm việc?

Có một tình huống mà mọi người làm thuật toán đều gặp: bạn dành nhiều giờ tìm một thuật toán đa thức cho một bài, thử hết mọi hướng ở phần C, và không ra. Câu hỏi tự nhiên là “mình

còn thiếu ý tưởng gì?”. Nhưng có một khả năng khác mà người mới thường không nghĩ tới: *có thể không ai biết cách giải nhanh bài này*, và việc bạn không nghĩ ra không phải do bạn kém.

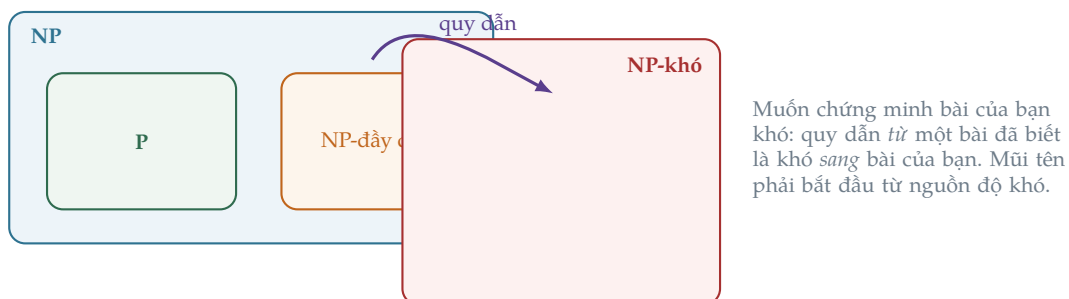
Chương này dạy cách phân biệt hai tình huống đó. Đây là kỹ năng có giá trị thực dụng rất cao — không phải vì nó giúp bạn giải thêm bài, mà vì nó giúp bạn *ngừng đúng lúc* và chuyển sang câu hỏi đúng: nếu không có lời giải nhanh chính xác, tôi sẵn sàng đánh đổi cái gì — tính tối ưu, tính tổng quát, hay thời gian chạy?

3.1 Tìm và kiểm: sự phân biệt sinh ra cả lý thuyết

Xét bài toán: cho một đồ thị và số k , có tồn tại một tập k đỉnh đôi một không kề nhau không? Tìm ra tập đó thì khó — không ai biết cách nào tốt hơn về cơ bản so với thử các tập con. Nhưng nếu ai đó *đưa* cho bạn một tập và bảo “đây là đáp án”, bạn kiểm tra trong vài giây: đếm số đỉnh, kiểm mọi cặp.

Đó chính là định nghĩa của lớp NP: các bài toán mà một lời giải, nếu được cho sẵn, kiểm chứng được nhanh. Lớp P là các bài mà bản thân việc *tìm* cũng nhanh. Rõ ràng P nằm trong NP. Câu hỏi P có bằng NP không — tức là “kiểm được nhanh thì có tìm được nhanh không” — là câu hỏi mở nổi tiếng nhất của ngành, và phần lớn giới nghiên cứu tin rằng câu trả lời là không.

Về lý do lý thuyết dùng bài toán quyết định thay vì bài toán tối ưu: đó là để có một khái niệm “kiểm chứng” sạch sẽ. Với bài quyết định, chứng cứ cho câu trả lời “có” là một cấu hình cụ thể. Với bài tối ưu, để kiểm “đây là giá trị tối ưu” bạn cần chứng minh không có gì tốt hơn — một việc khó hơn hẳn. May mắn là hai dạng liên hệ chặt với nhau qua đúng kỹ thuật ở chương C/19: giải bài tối ưu bằng cách nhị phân trên đáp án và gọi bài quyết định. Vì vậy phát biểu lý thuyết qua bài quyết định không làm mất tính tổng quát — đó là câu trả lời cho câu hỏi mở đầu thứ nhất.



Hình 3.1. Bản đồ các lớp. P nằm trong NP (tin là thực sự nhỏ hơn, nhưng chưa chứng minh được). NP-đầy đủ là các bài khó nhất *trong* NP; NP-khó rộng hơn và chứa cả những bài không thuộc NP, ví dụ các bài tối ưu tương ứng. Mũi tên tím nhắc lại chiều đúng của quy dẫn — lỗi phổ biến nhất khi lần đầu viết chứng minh độ khó.

3.2 Quy dẫn: công cụ để so độ khó

Ý tưởng trung tâm của chương: nếu tôi biến được bài A thành bài B một cách nhanh chóng, thì B *khó ít nhất bằng* A. Vì nếu B có lời giải nhanh, ghép với phép biến đổi ta được lời giải nhanh cho A.

Chiều của lập luận là chỗ mà gần như mọi người mới nhầm. Muốn chứng minh *bài của bạn khó*, bạn phải quy dẫn từ một bài đã biết là khó sang bài của bạn — tức là chỉ ra rằng bài của bạn đủ mạnh để mô phỏng bài khó kia. Quy dẫn theo chiều ngược lại — biến bài của bạn thành một bài khó đã biết — không chứng minh gì cả, vì ai cũng biến được mọi thứ thành một bài đủ tổng quát.

Đây là câu trả lời cho câu hỏi mở đầu thứ ba, và cũng là lỗi phổ biến nhất khi lần đầu viết một chứng minh độ khó.

Cấu trúc của một quy dẫn gồm ba phần: mô tả phép biến đổi từ dữ liệu của A thành dữ liệu của B; chứng minh phép biến đổi chạy trong thời gian đa thức; và chứng minh *cả hai chiều* của tương đương — A có nghiệm thì B có nghiệm, và ngược lại. Phần cuối là phần dễ bị làm ẩu nhất, và nó cũng là phần chứa nội dung thật.

Kết quả nền của cả lý thuyết là định lý Cook–Levin: bài toán thoả mãn công thức lôgic (SAT) là NP-đầy đủ. Ý nghĩa: nó là bài toán “khó nhất trong NP”, và từ nó có thể quy dẫn tới mọi bài NP khác. Sau đó Karp chứng minh 21 bài toán tổ hợp quen thuộc cũng NP-đầy đủ, biến một kết quả lôgic trừu tượng thành một danh mục dùng được.

3.3 Danh mục cần thuộc

Trong thực hành, bạn không chứng minh NP-khó từ đầu mà quy dẫn từ một bài trong danh mục. Bảng dưới là những bài hay dùng nhất, kèm dạng của chúng — vì nhận ra “bài của tôi có dạng giống bài này” là kỹ năng thật sự cần.

Bài toán	Dạng	Hay dùng để quy dẫn khi
3-SAT	Gán đúng/sai cho biến thoả mọi mệnh đề	Bài có cấu trúc “chọn một trong hai” kèm ràng buộc
Tập độc lập / Clique	Chọn nhiều đỉnh đôi một không kề / đôi một kề	Bài chọn nhiều đối tượng không xung khắc
Phủ đỉnh	Chọn ít đỉnh chạm mọi cạnh	Bài “chọn ít nhất để bao phủ”
Tô màu đồ thị	Gán k màu, hai đỉnh kề khác màu	Bài phân nhóm có ràng buộc xung khắc
Chu trình Hamilton / TSP	Đi qua mọi đỉnh đúng một lần	Bài về thứ tự đi qua toàn bộ
Ba lô / Phân hoạch	Chọn tập con đạt tổng mục tiêu	Bài có ràng buộc tổng số
Phủ tập hợp	Chọn ít tập nhất phủ hết	Bài phân bổ tài nguyên bao phủ
Đóng thùng	Xếp đồ vào ít thùng nhất	Bài lập lịch, phân bổ dung lượng

Một lưu ý quan trọng và hay bị bỏ: *nhiều bài NP-khó trở nên dễ trên lớp dữ liệu hẹp hơn*. Phủ đỉnh là NP-khó trên đồ thị tổng quát nhưng đa thức trên đồ thị hai phía (định lý König, C/18) và trên cây. Tô màu là NP-khó với ba màu nhưng đa thức với hai màu (kiểm tra đồ thị hai phía). Tập độc lập là NP-khó nói chung nhưng đa thức trên cây bằng quy hoạch động. Vì vậy khi gặp một bài trông giống bài NP-khó, câu hỏi tiếp theo phải là: *cấu trúc dữ liệu của tôi có đặc biệt không?*

3.4 Bốn cách sống chung

Đây là mục thực dụng nhất chương. Khi đã xác định bài là NP-khó, bạn có bốn hướng, và việc chọn hướng nào phụ thuộc vào cái bạn sẵn sàng từ bỏ.

Hướng 1 — giữ tính chính xác, từ bỏ tính đa thức. Dùng thuật toán mũ nhưng thông minh: quy hoạch động trên mặt nạ bit cho $O(2^n n^2)$ thay vì $O(n!)$ (C/16); nhánh cận với cận chặt (C/13); gấp nhau ở giữa cho $2^{n/2}$. Hướng này đúng khi n nhỏ, và câu hỏi trở thành “hàm mũ với cơ số bao nhiêu”.

Hướng 2 — giữ tính đa thức, từ bỏ tính tối ưu. Thuật toán xấp xỉ có bảo đảm: tham lam cho phủ tập hợp với hệ số $\ln n$, thuật toán 2-xấp xỉ cho phủ đỉnh, bảo đảm $1 + 1/e$ cho hàm dưới mô-đun (C/15). Điểm quan trọng: đây là bảo đảm chứng minh được cho mọi dữ liệu, khác hẳn “thực nghiệm thấy tốt”.

Hướng 3 — giữ cả hai, từ bỏ tính tổng quát theo một tham số. Nhiều bài có lời giải $O(f(k) \cdot n^c)$ với k là một tham số nhỏ — kích thước lời giải, độ rộng cây của đồ thị, số ràng buộc vi phạm. Nếu k nhỏ trong dữ liệu thật thì thuật toán chạy nhanh dù bài là NP-khó. Hướng này gọi là tham số hoá, và nó rất hợp với thực tế vì dữ liệu thật hiếm khi là trường hợp xấu nhất.

Hướng 4 — giao cho bộ giải công nghiệp. Mã hoá bài toán thành SAT, quy hoạch nguyên, hoặc một bài toán ràng buộc, rồi dùng bộ giải sẵn có. Đây là câu trả lời cho câu hỏi mở đầu thứ tư: các bộ giải hiện đại xử lý được những trường hợp hàng triệu biến vì dữ liệu công nghiệp có rất nhiều cấu trúc ẩn, và bộ giải khai thác chúng bằng lan truyền ràng buộc và học từ xung đột (C/13). Chúng không phá vỡ độ khó lý thuyết; chúng chỉ giải tránh trường hợp xấu.

CẦN NHỚ

Điều quý nhất mà chương này cho bạn không phải khả năng chứng minh NP-khó mà là khả năng *nhận ra sớm*. Khi một bài trông giống một mục trong danh mục ở mục 3, hãy dừng lại năm phút để kiểm tra thay vì tìm tiếp lời giải đa thức. Nếu đúng là NP-khó, câu hỏi lập tức đổi từ “giải thế nào” sang “ n bao nhiêu, tôi cần chính xác hay gần đúng, và dữ liệu có cấu trúc đặc biệt gì”.

3.5 Ngoài NP: hai điều nên biết là có

Có những bài toán không giải được, chấm hết. Bài toán dừng — cho một chương trình và dữ liệu vào, hỏi nó có dừng không — không có thuật toán nào giải được, và điều này đã được chứng minh chứ không phải chưa ai tìm ra. Đây là kết quả của Turing năm 1936, và nó có hệ quả thực dụng: nhiều câu hỏi về phân tích chương trình — “đoạn mã này có bao giờ chạy không”, “hai chương trình này có tương đương không” — là không giải được trong trường hợp tổng quát. Đó là lý do các công cụ phân tích tĩnh luôn hoặc bỏ sót hoặc báo động giả; sự không hoàn hảo đó là bắt buộc về mặt toán học, không phải do công cụ kém.

Có những bài khó hơn NP. Các bài về trò chơi hai người thường thuộc lớp cao hơn, vì chúng có cấu trúc “tồn tại nước đi sao cho với mọi phản ứng của đối thủ...” lồng nhau nhiều tầng. Đây là lý do các trò chơi tổng quát hoá — cờ vây trên bàn $n \times n$ chẳng hạn — khó hơn hẳn các bài tối ưu thông thường.

3.6 Gốc rễ: từ câu hỏi của Hilbert tới danh mục của Karp

Dòng suy nghĩ dẫn tới chương này bắt đầu từ toán học thuần túy. Đầu thế kỷ 20, Hilbert đặt ra bài toán quyết định: có tồn tại một thủ tục cơ học để xác định một mệnh đề toán học là đúng hay sai không? Năm 1936, Turing trả lời là không — và để trả lời được, ông phải *định nghĩa* thế nào là “thủ tục cơ học”, tạo ra máy Turing. Nghĩa là khái niệm thuật toán được hình thức hoá không phải để chế tạo máy tính mà để chứng minh một giới hạn.

Bước tiếp theo là phân biệt giữa “giải được” và “giải được trong thời gian chấp nhận”. Ý tưởng lấy *thời gian đa thức* làm ranh giới của cái “khả thi” được Edmonds phát biểu rõ vào năm 1965, khi ông trình bày một thuật toán ghép đôi và nhấn mạnh rằng nó chạy đa thức chứ không chỉ hữu hạn. Đây là một lựa chọn quy ước có tranh cãi — một thuật toán $O(n^{100})$ là đa thức nhưng

vô dụng — nhưng nó đã tỏ ra rất hữu ích trong thực tế, vì các thuật toán đa thức tự nhiên hầu như luôn có số mũ nhỏ.

Năm 1971, Cook chứng minh rằng SAT là bài khó nhất trong NP; Levin chứng minh kết quả tương đương một cách độc lập ở Liên Xô. Nhưng chính bài báo của Karp năm 1972 mới là thứ làm lý thuyết này lan rộng: ông chỉ ra 21 bài toán tổ hợp quen thuộc — những bài mà giới vận trù học và tổ hợp đã vật lộn nhiều năm — đều NP-đầy đủ. Tác động về mặt tâm lý rất lớn: hoá ra nhiều người đã thất bại trong việc tìm thuật toán nhanh cho những bài khác nhau, nhưng tất cả họ đang vấp phải *cùng một bức tường*.

Đó cũng là bài học lớn nhất của chương, và nó thuộc về phương pháp làm việc hơn là về lý thuyết: kết quả về độ khó không phải tin xấu mà là thông tin. Nó chuyển câu hỏi từ “tôi chưa đủ giỏi” sang “bài này đòi hỏi một loại đánh đổi”, và nó cho phép cả một cộng đồng ngừng tìm cái không có và bắt đầu xây dựng cái có — xấp xỉ, tham số hoá, bộ giải. Toàn bộ mục 4 của chương này là thành quả của sự chuyển hướng đó.

I Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
1928–1936 — Hilbert, Turing	Có thủ tục cơ học nào quyết định mọi mệnh đề toán học không?	Không — và để chứng minh, Turing phải định nghĩa “thuật toán”; máy Turing ra đời từ một kết quả <i>phủ định</i> .
1965 — Edmonds	Cần một ranh giới rõ ràng giữa “giải được” và “giải được trong thực tế”.	Thời gian đa thức làm chuẩn cho tính khả thi; một quy ước có tranh cãi nhưng hữu ích.
1971 — Cook; Levin (độc lập)	Có bài toán nào “khó nhất” trong lớp các bài kiểm chứng nhanh không?	SAT là NP-đầy đủ; mọi bài trong NP quy dẫn về nó.
1972 — Karp	Kết quả của Cook là logic trừu tượng, giới tổ hợp chưa thấy liên quan.	21 bài toán tổ hợp quen thuộc đều NP-đầy đủ; nhiều thất bại riêng lẻ hoá ra là cùng một bức tường.
Thập niên 1970–90 — xấp xỉ và tham số hoá	Bài NP-khó vẫn phải giải trong thực tế.	Thuật toán xấp xỉ có bảo đảm; lý thuyết phức tạp tham số hoá tách “khó theo n ” khỏi “khó theo k ”.
Cuối thập niên 1990–nay — bộ giải công nghiệp	Các bài logic và tối ưu thật có hàng triệu biến.	Bộ giải SAT và quy hoạch nguyên khai thác cấu trúc ẩn; NP-khó không còn đồng nghĩa với “không giải được trong thực tế”.

I 3.7 Liên hệ ngang

I Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Mật mã	Độ khó làm tài nguyên	Đảo ngược hoàn toàn quan hệ với độ khó: ở đây ta cần bài toán ngược khó. Điểm tinh tế: mật mã cần khó ở <i>trường hợp trung bình</i> , còn NP-đầy đủ chỉ nói về trường hợp xấu nhất — nên NP-đầy đủ không tự động dùng làm nền mật mã được.
Vận trù học	Bộ giải quy hoạch nguyên trong logistics, lập lịch bay	Hướng 4 ở mục 4, ở quy mô công nghiệp. Các bài lập lịch thật là NP-khó nhưng được giải hằng ngày — vì dữ liệu thật có cấu trúc và vì người ta chấp nhận lời giải tốt thay vì tối ưu tuyệt đối.

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu (tiếp)
Kiểm chứng phần mềm và phần cứng	Bộ giải SAT/SMT, phân tích tĩnh	Hai liên hệ: hướng 4 dùng SAT làm công cụ; và kết quả không giải được của Turing giải thích vì sao mọi công cụ phân tích tĩnh đều phải chấp nhận bỏ sót hoặc báo động giả.
Sinh học tính toán	Gấp protein, lắp ráp bộ gen, cây phát sinh loài	Nhiều bài trong nhóm này là NP-khó, và ngành này sống bằng đúng bốn hướng ở mục 4 — đặc biệt là heuristic có kiểm chứng thực nghiệm và các trường hợp riêng có cấu trúc.
Kinh tế học	Độ phức tạp của việc tìm điểm cân bằng	Câu hỏi “con người có thật sự đạt tới cân bằng không” có một chiều tính toán: nếu tìm cân bằng là khó về mặt tính toán thì mô hình giả định mọi người tính được nó là đáng ngờ. Đây là ví dụ độ phức tạp làm thay đổi một ngành khác.
Trò chơi	Sudoku, dò mìn, xếp gạch tổng quát hoá đều NP-khó	Cách kiểm tra trực giác tốt: nếu một trò chơi giải đồ cảm giác “phải thử rồi quay lui” thì rất có thể phiên bản tổng quát của nó NP-khó. Các trò hai người thường còn khó hơn NP.
Robot và SLAM	Ghép dữ liệu, chọn tập ràng buộc nhất quán tối đa	Bài “chọn tập lớn nhất các quan sát nhất quán với nhau” có dạng tập độc lập — NP-khó nói chung. Đó là lý do thực tế người ta dùng RANSAC và các heuristic ($C/20$) thay vì tìm nghiệm tối ưu.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì bài quyết định cho một khái niệm “chứng cứ” sạch sẽ: câu trả lời “có” luôn kèm một cấu hình cụ thể mà ta kiểm được nhanh. Với bài tối ưu, để kiểm “đây là giá trị tối ưu” bạn phải chứng minh không có gì tốt hơn — một nghĩa vụ nặng hơn hẳn. Việc phát biểu qua dạng quyết định không làm mất tính tổng quát, vì bài tối ưu quy về bài quyết định bằng cách nhai phân trên đáp án ($C/19$).
- A2. Vì $O(nW)$ là giả đa thức: nó đa thức theo giá trị W , nhưng W được biểu diễn bằng $\log W$ bit trong dữ liệu vào. Với $W = 2^{100}$, dữ liệu vào vẫn ngắn còn thuật toán cần 2^{100} bước. Độ phức tạp luôn đo theo kích thước biểu diễn, nên $O(nW)$ là mũ theo thước đo đó. Điều thú vị là điều này gợi ý một cách phân loại tinh hơn: các bài NP-khó có lời giải giả đa thức thì “nhẹ” hơn các bài NP-khó cả khi số nhỏ, ví dụ bài chu trình Hamilton.
- A3. Từ bài đã biết là khó sang bài của bạn: bạn phải chỉ ra rằng bài của bạn đủ mạnh để mô phỏng một bài khó. Chiều ngược lại vô nghĩa vì việc biến bài của bạn thành một bài tổng quát đã biết là khó không nói gì cả — mọi bài đều nhúng được vào một bài đủ tổng quát. Cách nhớ: quy dẫn chuyển độ khó theo chiều mũi tên, nên mũi tên phải bắt đầu từ nguồn độ khó.
- A4. Vì NP-khó nói về trường hợp xấu nhất, còn dữ liệu công nghiệp hầu như không bao giờ là trường hợp xấu nhất — nó có rất nhiều cấu trúc: ràng buộc lặp lại, biến gần như độc lập, đối xứng. Bộ giải hiện đại khai thác chính những cấu trúc đó bằng lan truyền ràng buộc, học mệnh đề từ xung đột, và phá đối xứng ($C/13$). Chúng không mâu thuẫn với lý thuyết: vẫn tồn tại những trường hợp nhỏ mà chúng không giải nổi, và người ta dùng đúng những trường hợp đó để so sánh bộ giải.
- A5. Bốn thay đổi cụ thể. Một, ngừng tìm lời giải đa thức chính xác — tiết kiệm hàng giờ hoặc hàng ngày. Hai, đọc lại giới hạn của bài: nếu n nhỏ thì tìm thuật toán mũ tốt. Ba, hỏi dữ liệu có cấu trúc đặc biệt không — cây, đồ thị hai phía, tham số nhỏ — vì rất nhiều bài NP-khó trở nên dễ trên lớp hẹp hơn. Bốn, nếu tất cả đều không, chọn một trong bốn hướng ở mục 4 và nói rõ mình đang đánh đổi cái gì.

NÓI LẠI THẬT ĐƠN GIẢN

Có một sự khác nhau rất lớn giữa việc tìm ra một lời giải và việc kiểm tra một lời giải người khác đưa cho.

Hãy nghĩ tới một câu đố ghép hình khó. Tự ghép thì có thể mất cả buổi. Nhưng nếu ai đó đưa cho bạn một cách ghép và bảo “xong rồi đây”, bạn chỉ mất một phút để kiểm xem có đúng không. Rất nhiều bài toán quan trọng có tính chất này: kiểm thì nhanh, tìm thì không ai biết cách nào nhanh. Câu hỏi lớn nhất của ngành khoa học máy tính chính là: có phải cứ kiểm được nhanh thì tìm cũng được nhanh? Chưa ai trả lời được, và hầu hết mọi người tin là không.

Điều thực dụng hơn là chuyện sau. Người ta phát hiện ra rằng rất nhiều bài toán khó khác nhau — xếp lịch, chia đồ, đi qua mọi thành phố một lần, tô màu bản đồ với ràng buộc — thực chất là cùng một bài khoắc áo khác nhau. Nếu ai giải nhanh được một cái thì giải nhanh được tất cả. Điều đó nghe như tin xấu, nhưng thật ra là tin rất tốt: nó có nghĩa là khi bạn nhận ra bài của mình thuộc nhóm đó, bạn biết ngay rằng không phải mình kém, mà là cả thế giới cũng chưa ai làm được.

Và khi biết vậy, câu hỏi đổi hẳn. Thay vì hỏi “giải thế nào”, bạn hỏi: dữ liệu của tôi có nhỏ không (nếu nhỏ thì cứ thử hết một cách thông minh); tôi có chấp nhận lời giải gần đúng không (có những cách gần đúng kèm bảo đảm hẳn hoi); dữ liệu của tôi có đặc điểm gì riêng không (rất nhiều bài khó trở nên dễ khi dữ liệu có dạng cây hoặc chia được thành hai nhóm). Đó mới là các câu hỏi dẫn tới lời giải dùng được.

Một câu: *Biết bài toán là khó không phải thất bại — đó là lúc câu hỏi đổi từ “giải thế nào” sang “đánh đổi cái gì”.*

Chỗ không rút gọn được: Việc P có bằng NP hay không vẫn là câu hỏi mở; mọi điều ở trên dựa trên niềm tin phổ biến rằng chúng khác nhau, chứ không dựa trên chứng minh.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● P và NP theo cách hiểu thực dụng	P: tìm được nhanh. NP: <i>kiểm</i> được nhanh nếu có lời giải cho sẵn. NP không có nghĩa là “không đa thức”.
●●● Chiều của quy dẫn	Muốn chứng minh bài của bạn khó: quy dẫn <i>từ</i> bài đã biết là khó <i>sang</i> bài của bạn. Chiều ngược lại không chứng minh gì.
●●● Ba phần của một quy dẫn	Phép biến đổi; chứng minh nó đa thức; chứng minh tương đương <i>cả hai chiều</i> .
●●● Giá đa thức	Đa thức theo <i>giá trị</i> nhưng mũ theo <i>số bit</i> biểu diễn; ba lô $O(n^W)$ là ví dụ. Không mâu thuẫn với NP-khó.
●●● Bốn cách sống chung	Mũ thông minh (giữ chính xác); xấp xỉ có bảo đảm (giữ đa thức); tham số hoá $O(f(k)n^c)$ (giữ cả hai, hẹp theo k); bộ giải công nghiệp.
●●● Câu hỏi sau khi biết bài NP-khó	n bao nhiêu; cần chính xác hay gần đúng; dữ liệu có cấu trúc đặc biệt gì (cây, hai phía, tham số nhỏ).
●● Danh mục quy dẫn hay dùng	3-SAT, tập độc lập / clique, phủ đỉnh, tô màu, Hamilton/TSP, ba lô/phân hoạch, phủ tập hợp, đóng thùng.
●● Bài khó trở nên dễ khi nào	Trên cây, trên đồ thị hai phía, khi tham số nhỏ, khi số màu bằng 2 — luôn kiểm cấu trúc dữ liệu trước khi bỏ cuộc.
●● Cook–Levin và Karp	SAT là NP-đầy đủ (1971); Karp (1972) đưa 21 bài tổ hợp quen thuộc vào cùng lớp, biến lý thuyết thành công cụ dùng được.
●● Bài toán dừng	Không giải được — đã chứng minh, không phải chưa tìm ra. Hệ quả: phân tích tính buộc phải bỏ sót hoặc báo động giả.
●● Vì sao bộ giải công nghiệp mạnh	NP-khó nói về trường hợp xấu nhất; dữ liệu thật có cấu trúc, và bộ giải khai thác nó bằng lan truyền ràng buộc và học từ xung đột.
● NP-đầy đủ và mật mã	NP-đầy đủ là độ khó <i>trường hợp xấu nhất</i> , còn mật mã cần khó ở <i>trường hợp trung bình</i> — nên không dùng trực tiếp làm nền mật mã được.

●●● phải thuộc ●● nên nắm ● biết là có

Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Làm sao tôi nghi ngờ một bài là NP-khó ngay khi đọc đề?

| Ba dấu hiệu thực dụng. Một, bài yêu cầu chọn một tập con tối ưu với các ràng buộc xung khắc giữa các phần tử — rất gần tập độc lập hoặc phủ đỉnh. Hai, bài yêu cầu một thứ tự đi qua toàn bộ đối tượng với chi phí — gần TSP. Ba, và đáng tin nhất trong thi đấu: giới hạn n rất nhỏ ($n \leq 20$ hoặc $n \leq 40$) trong khi bài trông như một bài tối ưu tổng quát — người ra đề đang nói với bạn rằng không có lời giải đa thức.

2. Nếu bài là NP-khó thì trong kỳ thi tôi làm gì?

| Đọc lại giới hạn — chúng luôn cho biết lời giải mong muốn. $n \leq 20$: quy hoạch động mặt nạ bit. $n \leq 40$: gặp nhau ở giữa. n lớn nhưng có cấu trúc cây hoặc đồ thị hai phía: có lời giải đa thức cho trường hợp riêng. Tổng giá trị nhỏ: quy hoạch động giả đa thức. Nếu không dấu hiệu nào khớp thì rất có thể bạn đã hiểu sai đề — hãy đọc lại, vì đề thi hiếm khi yêu cầu giải một bài NP-khó tổng quát ở quy mô lớn.

3. Chứng minh NP-khó có xuất hiện trong phòng vẫn không?

| Chứng minh đầy đủ thì hiếm, nhưng *nhận ra* thì có và nó tạo ấn tượng tốt. Nếu người phỏng vấn đưa một bài mà bạn nhận ra là biến thể của tập độc lập hay TSP, việc nói “bài này ở dạng tổng quát là NP-khó, nên tôi sẽ hỏi thêm về ràng buộc dữ liệu, hoặc đề xuất một thuật toán xấp xỉ với bảo đảm” cho thấy đúng loại tư duy mà họ tìm — biết dừng đúng lúc và biết đặt câu hỏi về đánh đổi.

4. “Xấp xỉ 2 lần” nghĩa chính xác là gì?

| Nghĩa là với *mọi* dữ liệu vào, kết quả của thuật toán không tệ hơn hai lần giá trị tối ưu — với bài cực tiểu thì không quá gấp đôi, với bài cực đại thì không dưới một nửa. Đây là bảo đảm chứng minh được, không phải quan sát thực nghiệm. Ví dụ đơn giản: với phủ đỉnh, cứ lấy cả hai đầu của một cạnh chưa bị phủ và lặp lại — kết quả không quá gấp đôi tối ưu, vì mọi cạnh được chọn đôi một không chung đỉnh và lời giải tối ưu phải chứa ít nhất một đầu của mỗi cạnh đó.

5. Tôi nghe nói “nếu $P = NP$ thì mọi thứ sụp đổ”. Có đúng không?

| Đúng một phần và hay bị nói quá. Nếu $P = NP$ với một thuật toán *thực dụng* thì phần lớn mật mã khoá công khai hiện nay sụp, và rất nhiều bài tối ưu trở nên dễ. Nhưng một chứng minh $P = NP$ với thuật toán $O(n^{100})$ thì gần như không thay đổi gì trong thực tế. Ngược lại, nếu chứng minh được $P \neq NP$ thì cũng không thay đổi công việc hằng ngày — ta đã hành xử như thể điều đó đúng từ lâu. Giá trị của câu hỏi này chủ yếu nằm ở việc hiểu bản chất của tính toán.

6. Lý thuyết tham số hoá khác gì với việc “chỉ giải bài nhỏ”?

| Khác ở chỗ nó tách *hai* đại lượng thay vì một. Thuật toán $O(2^k \cdot n)$ với k là kích thước lời giải chạy tốt trên đồ thị hàng triệu đỉnh miễn là lời giải cần tìm nhỏ — điều rất hay gặp trong thực tế, ví dụ tìm một tập nhỏ các nút hỏng trong mạng lớn. Đây là một cách nhìn tinh hơn về độ khó: “khó theo cái gì” quan trọng hơn “khó hay dễ”.

7. Có bài toán nào chưa biết là P hay NP-đầy đủ không?

| Có, và chúng thú vị vì hiếm. Bài đẳng cấu đồ thị — hai đồ thị có cùng cấu trúc không — chưa được xếp dứt khoát: chưa ai tìm được thuật toán đa thức, cũng chưa ai chứng minh NP-đầy đủ, và có kết quả cho thấy nó gần như chắc chắn không NP-đầy đủ. Bài phân tích thừa số cũng ở trạng thái tương tự. Sự tồn tại của những bài này là một trong các lý do người ta tin rằng cấu trúc bên trong NP phong phú hơn hai lớp P và NP-đầy đủ.

Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Viết một bộ kiểm chứng đa thức cho bài tập độc lập	Bài tập khái niệm	Mục tiêu là cảm nhận rõ khác biệt giữa “kiểm” và “tìm”.
●● Quy dẫn tập độc lập về phủ đỉnh và ngược lại	Bài kinh điển	Phần bù của tập độc lập là phủ đỉnh; chứng minh cả hai chiều.

Bài	Nguồn	Điều cần nhận ra (tiếp)
●● Quy dẫn 3-SAT về tô màu ba màu	Bài kinh điển	Dựng tiện ích cho mỗi mệnh đề; đây là bài tập chuẩn để học cấu trúc một quy dẫn.
●● Giải TSP chính xác với $n \leq 18$	Bài kinh điển	Quy hoạch động mặt nạ bit; ước lượng $2^n n^2$ trước khi cài.
●● Thuật toán 2-xấp xỉ cho phủ đỉnh, đo tỉ số thực tế	Bài tập thực nghiệm	So với tối ưu tìm bằng vét cạn trên đồ thị nhỏ; tỉ số thật thường tốt hơn 2 nhiều.
●●● Tập độc lập lớn nhất trên cây bằng quy hoạch động	Bài kinh điển	Bài NP-khó tổng quát nhưng đa thức trên cây — ví dụ trực tiếp cho lời khuyên ở mục 3.
●●● Thuật toán tham số hoá $O(2^k n)$ cho phủ đỉnh kích thước $\leq k$	[4]	Chọn một cạnh, một trong hai đầu phải thuộc lời giải; cây nhánh có độ sâu k .
●●● Mã hoá một bài xếp lịch thành SAT và giải bằng bộ giải	Bài tập ứng dụng	So thời gian với vét cạn tự viết; đây là cách nhanh nhất để hiểu hướng 4 ở mục 4.
●●● Tìm một bài trong công việc của bạn và xác định nó có NP-khó không	Bài tập tự do	Nếu có, hãy viết ra bốn hướng đối phó và chọn một, kèm lý do về đánh đổi.
●●● Chứng minh bài đóng thùng NP-khó bằng quy dẫn từ bài phân hoạch	Bài kinh điển	Quy dẫn ngắn nhưng phải viết đủ hai chiều; kiểm lại chiều mũi tên.

Kết chương và đọc tiếp

Chương này thay đổi cách bạn phân bổ thời gian nhiều hơn bất kỳ chương kỹ thuật nào. Ba điều đáng mang theo: sự phân biệt giữa tìm và kiểm, vì nó là hạt nhân của toàn bộ lý thuyết; chiều của quy dẫn, vì đó là lỗi phổ biến nhất; và bốn hướng đối phó ở mục 4, kèm câu hỏi “tôi đang đánh đổi cái gì”.

Chương cuối của phần D nhìn vào giới hạn từ một hướng khác. Chương này nói về những bài toán mà *chưa ai* tìm được lời giải nhanh; chương tiếp theo nói về những trường hợp ta *chứng minh được* rằng không thể nhanh hơn, và về những mô hình tính toán khác — dữ liệu chảy qua một lần, quyết định phải ra khi chưa biết tương lai, bộ nhớ chậm hơn hàng nghìn lần — nơi bảng xếp hạng thuật toán thay đổi hoàn toàn (D/24).

Nguồn

- [1] M. Agrawal, N. Kayal, and N. Saxena, “PRIMES Is in P,” *Annals of Mathematics*, vol. 160, no. 2, pp. 781–793, 2004.
- [2] A. Laaksonen. “CSES Problem Set.” (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).
- [3] J. R. Shewchuk, “Adaptive Precision Floating-Point Arithmetic and Fast Robust Geometric Predicates,” *Discrete & Computational Geometry*, vol. 18, pp. 305–363, 1997.
- [4] M. Cygan, F. V. Fomin, Ł. Kowalik, et al., *Parameterized Algorithms*. Springer, 2015.
- [5] R. Impagliazzo and R. Paturi, “On the Complexity of k-SAT,” *Journal of Computer and System Sciences*, vol. 62, no. 2, pp. 367–375, 2001.
- [6] A. Backurs and P. Indyk, “Edit Distance Cannot Be Computed in Strongly Subquadratic Time (Unless SETH Is False),” *SIAM Journal on Computing*, vol. 47, no. 3, 2018, bản hội nghị: STOC 2015.
- [7] A. Gajentaan and M. H. Overmars, “On a Class of $O(n^2)$ Problems in Computational Geometry,” *Computational Geometry: Theory and Applications*, vol. 5, no. 3, pp. 165–185, 1995.
- [8] M. Frigo, C. E. Leiserson, H. Prokop, et al., “Cache-Oblivious Algorithms,” in *Proceedings of the 40th Annual Symposium on Foundations of Computer Science*, 1999, pp. 285–297.
- [9] F. McSherry, M. Isard, and D. G. Murray, “Scalability! But at What COST?” In *15th Workshop on Hot Topics in Operating Systems (HotOS)*, 2015.
- [10] D. D. Sleator and R. E. Tarjan, “Self-Adjusting Binary Search Trees,” *Journal of the ACM*, vol. 32, no. 3, pp. 652–686, 1985.
- [11] N. Alon, Y. Matias, and M. Szegedy, “The Space Complexity of Approximating the Frequency Moments,” *Journal of Computer and System Sciences*, vol. 58, no. 1, pp. 137–147, 1999.
- [12] T. H. Cormen, C. E. Leiserson, R. L. Rivest, et al., *Introduction to Algorithms*, 4th ed. MIT Press, 2022.
- [13] P. Flajolet, É. Fusy, O. Gandouet, et al., “HyperLogLog: The Analysis of a Near-Optimal Cardinality Estimation Algorithm,” in *Conference on Analysis of Algorithms (AofA)*, 2007.

Cận dưới và các mô hình tính toán khác (Lower Bounds and Other Models)

PHỤ THUỘC A/03 cho mô hình chi phí và phân cấp bộ nhớ; A/05 cho lập luận đếm; D/23 cho mặt kia của độ khó.

TÓM TẮT

Chương trước nói về những bài toán mà *chưa ai* tìm được lời giải nhanh. Chương này nói về hai thứ khác: những trường hợp ta *chứng minh được* rằng không thể nhanh hơn, và những mô hình tính toán mà trong đó bảng xếp hạng thuật toán thay đổi hoàn toàn. Nửa đầu trình bày ba kỹ thuật chứng minh cận dưới — cây quyết định, lập luận đối thủ, và đếm thông tin — cùng lý do vì sao cận dưới lại khó chứng minh tới vậy. Nửa sau bỏ giả định lớn nhất của cả quyển sách: rằng dữ liệu nằm vừa trong bộ nhớ, đọc được nhiều lần, và máy chỉ có một lõi. Khi giả định đó sai — bộ nhớ ngoài, luồng dữ liệu, trực tuyến, song song — thì thuật toán tốt nhất đối hẳn. Nếu chỉ nhớ một điều: mô hình chi phí quyết định thuật toán, nên khi kết quả thực nghiệm mâu thuẫn với lý thuyết, thủ phạm gần như luôn là mô hình chứ không phải phép đo.

KIẾN THỨC CẦN CÓ TRƯỚC

- **cận dưới** — phát biểu rằng *mọi* thuật toán trong một mô hình nào đó đều cần ít nhất một lượng tài nguyên nhất định. Khác hẳn với việc phân tích chi phí của một thuật toán cụ thể.
- **cây quyết định** — mô hình trong đó thuật toán chỉ được đặt câu hỏi so sánh; mỗi lá là một kết quả khả dĩ, nên số lá bị chặn dưới bởi số kết quả.
- **lập luận đối thủ** — kỹ thuật chứng minh cận dưới bằng cách hình dung một đối thủ trả lời các câu hỏi của thuật toán một cách bất lợi nhất, miễn là vẫn nhất quán.
- **mô hình bộ nhớ ngoài** — tính chi phí theo số lần chuyển khối giữa bộ nhớ nhanh và chậm, thay vì theo số phép toán.
- **thuật toán luồng dữ liệu** — đọc dữ liệu một hoặc vài lượt, dùng bộ nhớ nhỏ hơn dữ liệu rất nhiều; thường phải chấp nhận kết quả xấp xỉ.
- **tỉ lệ cạnh tranh** — với thuật toán trực tuyến, tỉ số giữa chi phí của nó và chi phí của lời giải tối ưu biết trước toàn bộ tương lai.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao chứng minh “không thuật toán nào nhanh hơn” lại khó hơn nhiều so với việc tìm một thuật toán nhanh?
- Q2. Cận dưới $\Omega(n \log n)$ cho sắp xếp có mâu thuẫn với việc sắp xếp đếm chạy $O(n)$ không?
- Q3. Trong mô hình bộ nhớ ngoài, vì sao một thuật toán “chậm hơn” về số phép toán lại có thể nhanh hơn nhiều lần?
- Q4. Thuật toán trực tuyến không biết tương lai. Vậy ta đánh giá nó bằng thước đo nào cho công bằng?
- Q5. Vì sao một chương trình song song trên 32 lõi thường không nhanh gấp 32 lần?

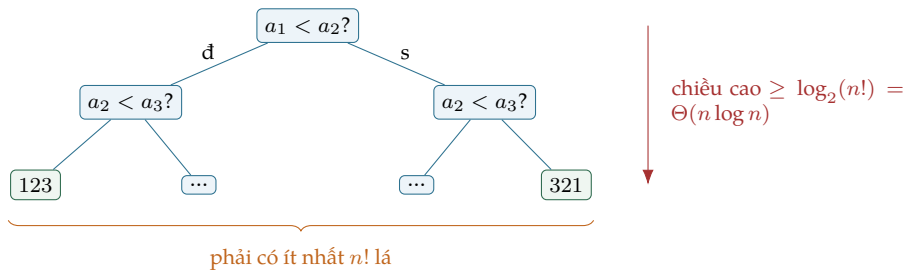
Toàn bộ quyển sách này, tới chương trước, đi theo một hướng: cho một bài toán, tìm thuật toán nhanh hơn. Chương này đi ngược lại và hỏi: *có điểm dừng không, và làm sao biết mình đã tới đó?*

Câu hỏi ấy có giá trị thực dụng lớn hơn vẻ ngoài học thuật của nó. Nếu bạn biết một bài toán cần ít nhất $\Omega(n \log n)$, bạn ngừng tìm lời giải tuyến tính. Nếu bạn biết mọi thuật toán so sánh đều cần $\Omega(n \log n)$ nhưng bài của bạn có thêm thông tin — khoá là số nguyên nhỏ — thì bạn biết chính xác phải phá vỡ giả định nào để đi nhanh hơn. Cận dưới không chỉ nói “dừng lại” mà còn nói *dừng ở đâu và vì sao*.

Nửa sau chương làm một việc khác nhưng cùng tinh thần: nó chỉ ra rằng nhiều kết luận trong các chương trước là hệ quả của một mô hình chi phí cụ thể, và khi mô hình đổi thì kết luận đổi theo.

4.1 Ba cách chứng minh cận dưới

Cách thứ nhất — cây quyết định. Nếu thuật toán chỉ được phép so sánh các phần tử, thì quá trình chạy của nó là một đường đi trên một cây nhị phân: mỗi nút là một phép so sánh, mỗi lá là một kết luận. Với bài sắp xếp n phần tử, số kết luận khả dĩ là $n!$ hoán vị, nên cây phải có ít nhất $n!$ lá, nên chiều cao ít nhất $\log_2(n!) = \Theta(n \log n)$. Đó là cận dưới nổi tiếng nhất trong ngành, và nó chỉ tồn tại ba câu để chứng minh — nhờ công thức Stirling ở chương A/05.



Hình 4.1. Cận dưới cho sắp xếp bằng cây quyết định. Mỗi phép so sánh chia đôi tập kết quả còn khả dĩ, mà số kết quả là $n!$, nên cây phải cao ít nhất $\log_2(n!)$. Điều bắt buộc phải nhớ kèm: kết luận này chỉ đúng *trong mô hình so sánh* — sắp xếp đếm dùng giá trị làm chỉ số nên nằm ngoài mô hình và không mâu thuẫn.

Điểm mấu chốt, và cũng là câu trả lời cho câu hỏi mở đầu thứ hai: cận này chỉ đúng *trong mô hình so sánh*. Sắp xếp đếm không so sánh phần tử với nhau mà dùng giá trị của chúng làm chỉ số mảng — một thao tác mà mô hình cây quyết định không cho phép. Không có mâu thuẫn nào; chỉ có hai mô hình khác nhau. Đây là bài học tổng quát: *mọi cận dưới đều kèm một mô hình, và bỏ qua mô hình là hiểu sai kết quả*.

Cách thứ hai — lập luận đối thủ. Hình dung một đối thủ không quyết định dữ liệu từ trước mà trả lời từng câu hỏi của thuật toán, luôn chọn câu trả lời bất lợi nhất, miễn là vẫn nhất quán với mọi câu đã trả lời. Nếu đối thủ giữ được ít nhất hai khả năng cho tới sau k câu hỏi thì thuật toán không thể kết luận sớm hơn $k + 1$. Ví dụ: tìm đồng thời cực đại và cực tiểu của n phần tử cần ít nhất $\lceil 3n/2 \rceil - 2$ phép so sánh, chứng minh bằng cách đếm lượng thông tin mà mỗi phép so sánh có thể cung cấp.

Cách thứ ba — đếm hoặc lập luận thông tin. Nếu kết quả cần phân biệt N khả năng và mỗi bước cho tối đa một bit thông tin thì cần ít nhất $\log_2 N$ bước. Đây là dạng tổng quát của cách thứ nhất, và nó cũng là ngôn ngữ chung với lý thuyết thông tin — lý do mà cận dưới cho sắp xếp và cận dưới cho nén dữ liệu có cùng hình thức.

4.2 Vì sao cận dưới khó

Câu hỏi mở đầu thứ nhất đáng được trả lời kỹ, vì nó giải thích tình trạng của cả ngành.

Để chứng minh một thuật toán *tồn tại*, bạn chỉ cần đưa ra một thuật toán — một đối tượng cụ thể. Để chứng minh *không thuật toán nào* nhanh hơn, bạn phải lập luận về *tất cả* các thuật toán khả dĩ, kể cả những cái chưa ai nghĩ ra. Đó là một mệnh đề toàn thể trên một tập vô hạn các đối tượng rất tự do, và ta có rất ít công cụ để nói điều gì đó về mọi thành viên của tập đó.

Vì vậy các cận dưới đã chứng minh được hầu như đều nằm trong những mô hình *hạn chế*: chỉ được so sánh, chỉ được truy cập theo một kiểu nhất định, chỉ có một lượt đọc. Trong mô hình tính toán tổng quát, chúng ta gần như không chứng minh được cận dưới nào đáng kể — và đó chính là lý do câu hỏi P đối với NP vẫn mở sau nửa thế kỷ. Câu chuyện Karatsuba ở chương C/14 là lời cảnh báo cụ thể: một nhà toán học hàng đầu phỏng đoán rằng nhân số cần $\Omega(n^2)$, và phỏng đoán đó bị bác bỏ trong một tuần. Trục giác về cận dưới rất dễ sai.

Một hướng hiện đại đáng biết là *độ phức tạp tính*: thay vì cố chứng minh cận dưới tuyệt đối, người ta chứng minh cận dưới *có điều kiện*, dạng “nếu bài toán A không thể giải nhanh hơn thì bài toán B cũng không”. Nhờ đó có những kết quả kiểu “khoảng cách chỉnh sửa không thể tính nhanh hơn đáng kể so với $O(n^2)$, trừ khi một giả thuyết được tin rộng rãi bị bác bỏ”. Đây là cách ngành này sống chung với sự khó của cận dưới: xây một mạng lưới các kết quả liên hệ với nhau thay vì chờ một chứng minh tuyệt đối.

4.3 Độ phức tạp tính: cận dưới có điều kiện, và cách dùng chúng

Mục trên kết thúc bằng một câu về “độ phức tạp tính”. Ý tưởng ấy đáng được nói kỹ hơn, vì nó là hướng sống động nhất của lý thuyết cận dưới hiện nay và vì nó có ích ngay cả cho người chỉ đi thi.

Cách đặt vấn đề là như sau. Ta không chứng minh nổi cận dưới tuyệt đối, vậy hãy chọn một *giả thuyết neo* mà cộng đồng tin, rồi quy dẫn để cho thấy “nếu bài toán của tôi nhanh hơn thì giả thuyết neo sụp”. Giả thuyết neo hay dùng nhất là **giả thuyết thời gian mũ mạnh** (*strong exponential time hypothesis*, SETH): không có $\varepsilon > 0$ nào để mọi bài k -SAT giải được trong $(2 - \varepsilon)^n$ — một phát biểu ra đời từ Impagliazzo và Paturi[5]. Giả thuyết neo thứ hai là bài 3SUM: cho n số, có ba số nào cộng lại bằng 0 không — tin rằng không có thuật toán $n^{2-\varepsilon}$, dù các cải tiến ở mức thừa số lôgarit thì đã có.

Kết quả đáng nhớ nhất của hướng này, ít nhất với người đọc quyển sách này, là của Backurs và Indyk[6]: nếu tính được khoảng cách chỉnh sửa của hai chuỗi độ dài n trong $O(n^{2-\varepsilon})$ thì SETH sai. Nói cách khác, bảng quy hoạch động $O(nm)$ mà bạn viết ở chương C/16 — một bài tập nhập môn — gần như chắc chắn là đã tối ưu. Ba chục năm người ta cố cải tiến nó và không ai thành công; bây giờ ta biết *vì sao*.

Nhánh 3SUM cho một câu chuyện song song trong hình học. Gajentaan và Overmars[7] chỉ ra cả một họ bài toán phẳng — có ba điểm nào thẳng hàng không, hợp của các tam giác có lỗ không, một số bài lập kế hoạch chuyển động — đều khó ít nhất bằng 3SUM. Điều đó giải thích một hiện tượng mà người thi đấu hay gặp: rất nhiều bài hình học “chỉ” có lời giải n^2 và không ai tìm được gì tốt hơn.

Giá trị thực hành nằm ở chỗ ít ai nói ra: cận dưới có điều kiện cho bạn *quyền dùng tìm*. Khi nhận ra bài mình đang vật lộn quy dẫn được về khoảng cách chỉnh sửa hay về 3SUM, câu hỏi đúng không còn là “làm sao nhanh hơn về mặt tiệm cận” mà chuyển thành ba câu khác: hằng số có giảm được không, đề bài có ràng buộc riêng nào phá được cấu trúc khó không (chuỗi ngắn, bảng chữ cái nhỏ, các điểm nằm trên lưới), và có chấp nhận được lời giải xấp xỉ không. Đó cũng chính

là bốn cách sống chung ở chương D/23, chỉ khác là ở đây ranh giới không phải giữa đa thức và mũ, mà giữa n^2 và $n^{2-\varepsilon}$.

4.4 Mô hình bộ nhớ ngoài: khi đơn vị chi phí là lần đọc khối

Chương A/03 đã cảnh báo rằng mô hình RAM coi mọi truy cập bộ nhớ là như nhau, còn thực tế thì có phân cấp. Mục này lấy nhận xét đó làm mô hình chính thức.

Trong mô hình bộ nhớ ngoài, ta đếm số lần chuyển *khối* dữ liệu giữa bộ nhớ chậm và nhanh, mỗi khối chứa B phần tử, và bộ nhớ nhanh chứa được M phần tử. Với mô hình này, sắp xếp không còn là $O(n \log n)$ mà là $O(\frac{n}{B} \log_{M/B} \frac{n}{B})$ lần chuyển khối — và cơ số của lôgarit là M/B , thường hàng trăm, nên số “tầng” rất ít. Đó là nền lý thuyết cho sắp xếp trộn nhiều đường, và cũng là lý do B-cây có nút to ($B/09$).

Điều đáng chú ý nhất, và là câu trả lời cho câu hỏi mở đầu thứ ba: trong mô hình này, một thuật toán làm nhiều phép toán hơn nhưng truy cập *tuần tự* có thể thắng một thuật toán ít phép toán hơn nhưng nhảy ngẫu nhiên — vì chi phí thật nằm ở việc kéo khối dữ liệu, không ở việc tính toán.

Cache-oblivious là một ý tưởng đẹp trong nhánh này[8]: thiết kế thuật toán tối ưu về số lần chuyển khối mà *không cần biết* B và M là bao nhiêu. Kỹ thuật chung là chia để trị đệ quy tới mọi tỉ lệ, để ở tầng nào của phân cấp bộ nhớ thì cũng có một mức đệ quy vừa khít. Nhờ vậy một chương trình chạy tốt trên mọi máy mà không phải tinh chỉnh tham số.

4.5 Ba mô hình bỏ giả định khác

Luồng dữ liệu. Dữ liệu chảy qua một lần và quá lớn để lưu; bạn có bộ nhớ nhỏ hơn nhiều bậc. Trong mô hình này, hầu hết bài toán chính xác trở nên bất khả thi, và người ta chuyển sang xấp xỉ: đếm số phần tử phân biệt bằng HyperLogLog, tìm phần tử xuất hiện nhiều bằng sơ đồ đếm, lấy mẫu đều bằng hồ chứa ($C/20$). Đây là mô hình của các hệ thống đo lường và phân tích nhật ký ở quy mô lớn.

Trực tuyến. Bạn phải quyết định ngay khi mỗi dữ liệu tới, không được xét lại, và không biết tương lai. Câu hỏi mở đầu thứ tư hỏi về thước đo, và câu trả lời là *tỉ lệ cạnh tranh*: so chi phí của bạn với chi phí của một lời giải tối ưu *biết trước toàn bộ tương lai*. Ví dụ kinh điển là bài thuê hay mua: mỗi ngày trượt tuyết bạn có thể thuê ván với giá 1 hoặc mua hẳn với giá B , và bạn không biết mình sẽ trượt bao nhiêu ngày; chiến lược “thuê B ngày rồi mua” có tỉ lệ cạnh tranh 2, và người ta chứng minh được không chiến lược tắt định nào tốt hơn. Cùng khung phân tích áp cho các chính sách thay thế bộ nhớ đệm: LRU có tỉ lệ cạnh tranh phụ thuộc kích thước bộ đệm, và phân tích đó giải thích vì sao nó là lựa chọn hợp lý dù không tối ưu.

Song song. Bỏ giả định một lõi. Thước đo chuẩn gồm hai đại lượng: *tổng công việc* — số phép toán nếu chạy tuần tự — và *chiều sâu* — độ dài chuỗi phụ thuộc dài nhất, tức thời gian nếu có vô hạn bộ xử lý. Thời gian thực tế bị chặn dưới bởi chiều sâu, nên một thuật toán có chuỗi phụ thuộc dài thì song song hoá bao nhiêu cũng vô ích.

Đây là câu trả lời cho câu hỏi mở đầu thứ năm, và nó có ba lớp lý do. Thứ nhất, phần tuần tự không loại bỏ được sẽ chi phối khi số lõi tăng — nếu 5% chương trình là tuần tự thì dù có vô hạn lõi cũng không nhanh hơn 20 lần. Thứ hai, chi phí điều phối: tạo luồng, đồng bộ, truyền dữ liệu đều tốn, và chúng tăng theo số lõi. Thứ ba, tranh chấp tài nguyên chung, đặc biệt là băng thông bộ nhớ — nhiều lõi cùng đọc thì bộ nhớ trở thành nút cổ chai. Kết quả thực nghiệm nổi tiếng của

McSherry và cộng sự[9] cho thấy nhiều hệ thống song song bị một chương trình đơn luồng viết cẩn thận đánh bại, đúng vì ba lý do trên.

CẦN NHỚ

Mô hình chi phí quyết định thuật toán. Bốn mô hình trong chương này — bộ nhớ ngoài, luồng, trực tuyến, song song — mỗi cái làm một họ thuật toán khác nhau trở thành tốt nhất. Khi bạn thấy một thuật toán “tốt hơn về lý thuyết” lại thua trong thực nghiệm, câu hỏi đầu tiên không phải “phép đo có sai không” mà là “tôi đang ở mô hình chi phí nào”.

4.6 Gốc rễ: từ cân đo tới các hệ thống dữ liệu lớn

Cận dưới cho sắp xếp thuộc loại kết quả xuất hiện sớm và tự nhiên: các bài toán “cân bao nhiêu lần thì tìm ra đồng xu giả” đã có trong toán giải trí từ lâu, và chúng chính là lập luận cây quyết định ở dạng dân dã. Việc hình thức hoá thành một cận dưới cho sắp xếp thuộc về giai đoạn đầu của ngành, và nó vẫn là ví dụ chuẩn để dạy khái niệm “cận dưới trong một mô hình”.

Phân tích cạnh tranh cho thuật toán trực tuyến ra đời năm 1985 trong cùng bài báo mà Sleator và Tarjan giới thiệu splay tree[10] — một sự trùng hợp đáng chú ý, vì cả hai ý tưởng đều xuất phát từ cùng một câu hỏi: làm sao đánh giá công bằng một thuật toán không có đảm bảo cho từng thao tác riêng lẻ. Phân tích khấu hao trả lời cho trường hợp offline; phân tích cạnh tranh trả lời cho trường hợp không biết tương lai. Bối cảnh thúc đẩy là rất thực tế: các chính sách quản lý bộ nhớ đệm trong hệ điều hành cần một tiêu chuẩn đánh giá, và “trường hợp xấu nhất” thuần túy thì quá bi quan vì mọi chính sách đều tệ trên dữ liệu thù địch.

Mô hình bộ nhớ ngoài được hình thức hoá vào cuối thập niên 1980 khi cơ sở dữ liệu trở thành ngành lớn và khoảng cách tốc độ giữa đĩa và bộ nhớ ngày càng rõ. Cache-oblivious ra đời năm 1999[8] với một động cơ rất thực dụng: các thư viện tối ưu theo tham số phần cứng phải viết lại cho mỗi đời máy, và người ta muốn thoát khỏi vòng lặp đó.

Thuật toán luồng dữ liệu có một mốc rõ ràng: bài báo của Alon, Matias và Szegedy giữa thập niên 1990[11] đặt nền cho việc ước lượng các đại lượng thống kê với bộ nhớ nhỏ hơn dữ liệu nhiều bậc. Động cơ ban đầu mang tính lý thuyết, nhưng chỉ một thập kỷ sau, khi lưu lượng mạng và nhật ký hệ thống vượt xa khả năng lưu trữ, các kỹ thuật đó trở thành hạ tầng của mọi hệ thống giám sát.

Điểm chung của bốn câu chuyện: mỗi mô hình mới ra đời khi một giả định của mô hình cũ trở nên sai trong thực tế. Đó là mẫu đáng nhớ nhất của chương, và nó gợi ý rằng sẽ còn những mô hình mới — năng lượng, tính toán trên dữ liệu mã hoá, hay phần cứng chuyên dụng — khi các ràng buộc mới trở nên chi phối.

Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
Giai đoạn đầu của ngành — cận dưới sắp xếp	Có thuật toán sắp xếp nhanh hơn $n \log n$ không?	Lập luận cây quyết định: $\log_2(n!) = \Theta(n \log n)$; và bài học “cận dưới luôn kèm một mô hình”.
1967 — Amdahl	Máy nhiều bộ xử lý có thực sự nhanh hơn tỉ lệ thuận không?	Phân tuần tự không loại bỏ được sẽ chi phối; đặt kỳ vọng thực tế cho tính toán song song.
1985 — Sleator & Tarjan	Đánh giá thế nào cho công bằng một thuật toán không biết tương lai?	Tỉ lệ cạnh tranh: so với lời giải tối ưu biết trước tương lai; ra đời cùng bài báo với splay tree và phân tích khấu hao.

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó (tiếp)
Cuối thập niên 1980 — mô hình bộ nhớ ngoài	Cơ sở dữ liệu lớn hơn bộ nhớ; đĩa chậm hơn hàng nghìn lần.	Đếm chi phí theo lần chuyển khối; giải thích B-cây và sắp xếp trộn nhiều đường.
Giữa thập niên 1990 — Alon, Matias, Szegedy	Ước lượng thống kê trên dòng dữ liệu quá lớn để lưu.	Nền của thuật toán luồng; một thập kỷ sau trở thành hạ tầng giám sát hệ thống.
1999 — Frigo và cộng sự	Thư viện tối ưu theo tham số phần cứng phải viết lại cho mỗi đời máy.	Cache-oblivious: tối ưu mà không cần biết kích thước khối và bộ nhớ đệm.
2015 — McSherry và cộng sự	Hệ thống phân tán “mở rộng tốt” có thật sự nhanh không?	Nhiều hệ thống bị một chương trình đơn luồng viết tốt đánh bại; nhắc rằng chi phí điều phối không xuất hiện trong phân tích tiệm cận.

4.7 Liên hệ ngang

Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Lý thuyết thông tin	Entropy và cận dưới cho nén	Cùng lập luận đếm: cần $\log_2 N$ bit để phân biệt N khả năng. Cận dưới sắp xếp và cận dưới nén là hai mặt của một kết quả.
Cơ sở dữ liệu	Sắp xếp ngoài, chỉ mục, tối ưu hoá truy vấn	Mô hình bộ nhớ ngoài là mô hình chính thức của ngành này. Bộ tối ưu truy vấn ước lượng số lần đọc khối chứ không ước lượng số phép toán — đúng tinh thần mục 3.
Hệ điều hành và mạng phân phối nội dung	Chính sách thay thế bộ nhớ đệm	Phân tích cạnh tranh cho biết một chính sách trực tuyến kém tối đa bao nhiêu lần so với biết trước tương lai — thước đo phù hợp hơn “trường hợp xấu nhất” thuần túy.
Hệ thời gian thực và robot	Ràng buộc thời hạn cho từng chu kỳ	Ở đây khâu hao không dùng được và cả tỉ lệ cạnh tranh cũng không đủ: cái cần là cận trên cho <i>mỗi</i> thao tác. Đây là một mô hình chi phí riêng mà sách giáo khoa thuật toán ít nói tới.
Tính toán trên GPU	Mô hình công việc và chiều sâu, hợp nhất truy cập bộ nhớ	Chiều sâu là ràng buộc thật: thuật toán có chuỗi phụ thuộc dài không tăng tốc được dù có hàng nghìn lõi. Ngoài ra kiểu truy cập bộ nhớ quyết định nhiều hơn số phép toán — lại là mô hình bộ nhớ ngoài ở quy mô nhỏ.
Hệ thống dữ liệu lớn	Xử lý luồng, cửa sổ thời gian, sơ đồ xấp xỉ	Ứng dụng trực tiếp của mục 4. Đánh đổi đặc trưng: chấp nhận sai số vài phần trăm để đổi lấy khả năng chạy được ở quy mô mà lời giải chính xác không khả thi.
Thiết bị nhúng, robot chạy pin	Năng lượng làm đơn vị chi phí	Một mô hình gần như vắng mặt trong sách giáo khoa thuật toán nhưng chi phối trong thực tế: thuật toán ít phép toán hơn nhưng truy cập bộ nhớ nhiều hơn có thể tốn nhiều năng lượng hơn.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1.** Vì hai loại mệnh đề có bản chất khác nhau. Chứng minh tồn tại thuật toán nhanh chỉ cần đưa ra *một* đối tượng cụ thể. Chứng minh cận dưới đòi lập luận về *mọi* thuật toán khả dĩ, kể cả những cái chưa ai nghĩ ra — một mệnh đề toàn thể trên một tập vô hạn rất tự do. Vì vậy các cận dưới đã có hầu như đều nằm trong mô hình hạn chế (chỉ so sánh, một lượt đọc), còn trong mô hình

tổng quát thì ta gần như không chứng minh được gì — đó cũng là lý do câu hỏi P đối với NP vẫn mở.

- A2. Không mâu thuẫn, vì hai kết quả nằm trong hai mô hình khác nhau. Cận $\Omega(n \log n)$ chỉ áp cho các thuật toán *chỉ được so sánh phần tử với nhau*; sắp xếp đếm không so sánh mà dùng giá trị làm chỉ số mảng, một thao tác nằm ngoài mô hình đó. Bài học tổng quát: mỗi cận dưới luôn kèm một mô hình, và muốn vượt qua nó thì phải phá vỡ đúng giả định của mô hình — ở đây là “chỉ được so sánh”.
- A3. Vì trong mô hình đó, đơn vị chi phí là *lần chuyển khối*, không phải phép toán. Đọc một khối 4 KB từ đĩa tốn như nhau dù bạn dùng một phần tử hay cả khối, nên thuật toán truy cập tuần tự tận dụng được toàn bộ khối, còn thuật toán nhảy ngẫu nhiên trả giá đầy đủ cho mỗi phần tử. Chênh lệch giữa hai kiểu truy cập có thể tới hàng nghìn lần — đủ để lấn át một khác biệt về số phép toán. Cùng hiện tượng xảy ra ở quy mô nhỏ với cache, và đó là lý do mảng thắng con trỏ (B/06).
- A4. Bằng tỉ lệ cạnh tranh: tỉ số giữa chi phí của thuật toán trực tuyến và chi phí của lời giải tối ưu *biết trước toàn bộ tương lai*. Thước đo này công bằng ở chỗ nó không đòi hỏi thuật toán phải làm điều bất khả thi, mà chỉ hỏi nó tệ hơn bao nhiêu lần so với một đối thủ được biết tương lai. Ví dụ bài thuê hay mua: chiến lược “thuê tới khi tổng tiền thuê bằng giá mua rồi mua” có tỉ lệ 2, và không chiến lược tắt định nào tốt hơn.
- A5. Ba lớp lý do. Một, phần tuần tự không loại bỏ được: nếu 5% chương trình không song song hoá được thì dù vô hạn lõi cũng chỉ nhanh tối đa 20 lần. Hai, chi phí điều phối — tạo luồng, đồng bộ, truyền dữ liệu — tăng theo số lõi và có thể vượt phần được tăng tốc. Ba, tài nguyên chung: nhiều lõi cùng đọc bộ nhớ làm bằng thông thành nút cổ chai. Ba lý do này giải thích kết quả thực nghiệm rằng nhiều hệ thống song song thua một chương trình đơn luồng được viết cẩn thận[9].

NÓI LẠI THẬT ĐƠN GIẢN

Chương này nói về hai chuyện, và cả hai đều là chuyện “biết mình đang chơi theo luật nào”. Chuyện thứ nhất: đôi khi ta chứng minh được rằng không ai có thể nhanh hơn một mức nào đó. Ví dụ dễ hình dung nhất: nếu bạn chỉ được phép hỏi “số này lớn hơn số kia không”, thì để sắp xếp n món bạn buộc phải hỏi cỡ $n \log n$ câu, vì mỗi câu hỏi chỉ chia đôi số khả năng còn lại, mà số cách sắp xếp thì rất lớn. Điều quan trọng phải nhớ kèm theo: kết luận đó chỉ đúng *với luật chơi* “chỉ được hỏi so sánh”. Nếu bạn được phép dùng giá trị của món đồ làm địa chỉ ngăn tủ thì luật đã khác và bạn có thể nhanh hơn. Rất nhiều tranh cãi kiểu “sách nói không thể nhanh hơn mà tôi làm được” đến từ việc quên mất luật chơi.

Chuyện thứ hai còn thực dụng hơn: gần như toàn bộ quyển sách này ngầm giả định rằng dữ liệu nằm gọn trong bộ nhớ, đọc lại bao nhiêu lần cũng được, và máy chỉ làm một việc một lúc. Khi một trong ba giả định đó sai, bảng xếp hạng thuật toán đảo lộn.

Nếu dữ liệu nằm trên đĩa, thứ đắt không còn là phép tính mà là mỗi lần với tay xuống kho lấy một thùng — nên thuật toán tốt là thuật toán lấy ít thùng và dùng hết mọi thứ trong thùng. Nếu dữ liệu chảy qua một lần như nước và bạn chỉ có một cái ca nhỏ, bạn phải từ bỏ câu trả lời chính xác và chuyển sang ước lượng. Nếu bạn phải quyết định ngay mà chưa biết tương lai, cách đánh giá công bằng là so với một người biết trước mọi chuyện — và bạn chấp nhận tệ hơn họ vài lần. Còn nếu bạn có nhiều lõi thì hãy nhớ: chuỗi việc phải làm nối tiếp nhau sẽ quyết định thời gian, chứ không phải tổng số việc.

Một câu: Mọi kết luận về “thuật toán nào tốt hơn” đều là kết luận trong một luật chơi; đổi luật thì đổi câu trả lời.

Chỗ không rút gọn được: Chứng minh cận dưới trong mô hình tính toán tổng quát vẫn nằm ngoài khả năng hiện nay; đó là lý do nhiều câu hỏi lớn còn mở.

Thuật ngữ / câu hỏi	Trả lời
●●● Cận dưới luôn kèm mô hình	$\Omega(n \log n)$ cho sắp xếp chỉ đúng trong mô hình <i>so sánh</i> ; sắp xếp đếm không mâu thuẫn vì nó dùng giá trị làm chỉ số.
●●● Ba kỹ thuật cận dưới	Cây quyết định (số lá $\geq$ số kết quả); lập luận đối thủ (giữ nhiều khả năng càng lâu càng tốt); đếm thông tin ($\log_2 N$ bit).
●●● Vì sao cận dưới khó	Phải lập luận về <i>mọi</i> thuật toán khả dĩ, kể cả chưa ai nghĩ ra — mệnh đề toàn thể trên tập vô hạn tự do.
●●● Mô hình bộ nhớ ngoài	Đếm số lần chuyển khối, không đếm phép toán; sắp xếp là $O(\frac{n}{B} \log_{M/B} \frac{n}{B})$ lần chuyển. Giải thích B-cây và trộn nhiều đường.
●●● Hai đại lượng của song song	Tổng công việc và chiều sâu (chuỗi phụ thuộc dài nhất). Thời gian bị chặn dưới bởi chiều sâu, dù có bao nhiêu lõi.
●●● Ba lý do song song không tăng tuyến tính	Phản tuần tự chi phối; chi phí điều phối tăng theo số lõi; tranh chấp băng thông bộ nhớ.
●● Tỷ lệ cạnh tranh	So chi phí của thuật toán trực tuyến với lời giải tối ưu biết trước tương lai; bài thuê hay mua có tỉ lệ 2 và đó là tối ưu cho chiến lược tất định.
●● Mô hình luồng dữ liệu	Một lượt đọc, bộ nhớ nhỏ hơn dữ liệu nhiều bậc; hầu hết bài chính xác bất khả thi nên chuyển sang xấp xỉ (HyperLogLog, sơ đồ đếm, lấy mẫu hồ chứa).
●● Cache-oblivious	Tối ưu số lần chuyển khối mà không cần biết B và M ; đạt được bằng chia để trị đệ quy tới mọi tỉ lệ.
●● Độ phức tạp tinh	Cận dưới <i>có điều kiện</i> : “nếu bài A không nhanh hơn được thì B cũng không”; cách sống chung với việc cận dưới tuyệt đối quá khó.
●● Khi lý thuyết mâu thuẫn thực nghiệm	Nghi mô hình chi phí trước, đừng nghi phép đo — bạn có thể đang ở mô hình khác với mô hình mà kết quả lý thuyết giả định.
● Hệ thời gian thực	Cần cận trên cho <i>mỗi</i> thao tác; khấu hao và tỉ lệ cạnh tranh đều không đủ.
●● Cận dưới có điều kiện	Neo vào SETH hoặc 3SUM rồi quy dẫn: khoảng cách chính sửa không thể tính trong $O(n^{2-\epsilon})$ trừ khi SETH sai. Giá trị thực hành là biết khi nào nên ngừng tìm thuật toán nhanh hơn.

●●● phải thuộc ●● nên nắm ● biết là có

I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Cận dưới có ích gì cho công việc hằng ngày của tôi?

| Chủ yếu ở hai chỗ. Thứ nhất, nó cho bạn biết khi nào nên ngừng tìm — nếu bài của bạn quy về sắp xếp thì đừng tìm lời giải tuyến tính trong mô hình so sánh. Thứ hai, và hữu ích hơn, nó chỉ ra *giả định nào cần phá* để đi nhanh hơn: khoá có phải số nguyên nhỏ không (thì dùng sắp xếp đếm), dữ liệu có gần như đã sắp không (thì có thuật toán thích ứng), bạn có chấp nhận kết quả xấp xỉ không. Cận dưới không chỉ nói “không được” mà nói “không được *trong luật này*”.

2. Làm sao biết chương trình của tôi đang bị chặn bởi bộ nhớ chứ không phải bởi tính toán?

| Hai kiểm tra rẻ. Một: giảm kích thước dữ liệu xuống mức vừa trong bộ nhớ đệm và xem thời gian trên mỗi phần tử có giảm mạnh không — nếu có, bạn đang bị chặn bởi bộ nhớ. Hai: thay đổi *thứ tự truy cập* mà giữ nguyên số phép toán, ví dụ duyệt ma trận theo hàng thay vì theo cột; nếu thời gian đổi nhiều lần thì chi phí thật nằm ở truy cập. Đây là hai thí nghiệm nên làm trước khi tối ưu bất cứ thứ gì (E/26).

3. Thuật toán trực tuyến có xuất hiện trong bài tập không, hay chỉ là lý thuyết?

| Có, dưới hai dạng. Dạng thứ nhất là các bài yêu cầu trả lời truy vấn ngay mà không được xử lý ngoại tuyến — lúc đó các kỹ thuật như thuật toán Mo (B/10) không dùng được và bạn phải chọn cấu trúc dữ liệu khác. Dạng thứ hai là các bài mô phỏng chính sách quyết định, ví dụ quản lý bộ đệm.

Ngoài thi đấu thì đây là mô hình rất thực: mọi hệ thống phải quyết định khi chưa biết yêu cầu tiếp theo.

4. Bộ nhớ đệm LRU có tối ưu không?

| Không tối ưu, và người ta biết chính xác nó kém bao nhiêu. Chính sách tối ưu là loại bỏ trang sẽ được dùng lại xa nhất trong tương lai — không cài được vì cần biết tương lai. LRU có tỉ lệ cạnh tranh bằng kích thước bộ đệm trong trường hợp xấu nhất, nhưng trong thực tế nó chạy tốt vì các mẫu truy cập thật có tính cục bộ thời gian. Đây là ví dụ điển hình cho việc phân tích trường hợp xấu nhất quá bi quan, và là lý do các phân tích tinh hơn ra đời.

5. Nếu tôi muốn tăng tốc bằng song song, nên bắt đầu từ đâu?

| Bắt đầu bằng việc tính *chiều sâu* của thuật toán: nếu nó có một chuỗi phụ thuộc dài thì song song hoá không cứu được và bạn cần đổi thuật toán trước. Sau đó chọn nơi song song hoá ở mức thô nhất có thể — các nhánh trên cùng của chia để trị, các phần tử độc lập của một vòng lặp lớn — vì song song ở mức mịn thường bị chi phí điều phối nuốt hết. Cuối cùng, luôn đo trước và sau; kết quả thường khác dự đoán, đúng như bài học COST.

6. Thuật toán xấp xỉ trong mô hình luồng đáng tin tới đâu?

| Chúng đi kèm bảo đảm dạng “sai số tương đối không quá ε với xác suất ít nhất $1 - \delta$ ”, và bạn chọn ε, δ bằng cách trả thêm bộ nhớ. Đây là bảo đảm chứng minh được, không phải quan sát thực nghiệm — nên chúng đáng tin theo đúng nghĩa đã phát biểu. Điều cần cẩn thận trong thực tế: các bảo đảm này thường giả định các hàm băm đủ ngẫu nhiên và các phần tử của luồng không phụ thuộc vào lựa chọn ngẫu nhiên của bạn ($C/20$).

7. Có mô hình chi phí nào đang trở nên quan trọng mà sách giáo khoa chưa nói nhiều?

| Ít nhất hai. Năng lượng: trên thiết bị chạy pin và trong trung tâm dữ liệu, joule mỗi thao tác là ràng buộc thật, và nó không tỉ lệ với số phép toán — truy cập bộ nhớ tốn năng lượng hơn phép tính rất nhiều. Và tính toán trên dữ liệu được bảo vệ: khi phải tính trên dữ liệu mã hoá hoặc phân tán mà không lộ thông tin, chi phí của một phép toán thay đổi hoàn toàn, và các thuật toán tối ưu cho mô hình đó rất khác. Cả hai đều là ví dụ cho mẫu ở mục 6: mô hình mới ra đời khi một ràng buộc mới trở nên chi phối.

Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Chứng minh cận dưới $\Omega(n \log n)$ cho sắp xếp so sánh	[12]	Cây quyết định cộng công thức Stirling; ba câu là đủ.
● Chứng minh cần ít nhất $n - 1$ phép so sánh để tìm cực đại	Bài kinh điển	Lập luận đối thủ: mỗi phép so sánh loại tối đa một ứng viên.
●● Tìm đồng thời cực đại và cực tiểu với $\lceil 3n/2 \rceil - 2$ phép so sánh	Bài kinh điển	So từng cặp trước rồi mới so với kỷ lục; chứng minh cận dưới bằng lập luận đối thủ.
●● Sắp xếp một tệp lớn hơn bộ nhớ	Bài tập hệ thống	Trộn nhiều đường với heap (B/07); đo số lần đọc/ghi và so với dự đoán của mô hình bộ nhớ ngoài.
●● Đếm số phần tử phân biệt trong luồng bằng bộ nhớ nhỏ	[13]	Cài HyperLogLog đơn giản; đo sai số thực tế trên dữ liệu vài triệu phần tử.
●● Mô phỏng LRU và chính sách tối ưu trên cùng dãy truy cập	Bài tập thực nghiệm	Đo tỉ số trượt bộ đệm; đối chiếu với khái niệm tỉ lệ cạnh tranh.
●●● Nhân ma trận theo khối và đo hiệu ứng cache	Bài tập thực nghiệm	So ba cách: ngây thơ, đối thủ tự vòng lặp, chia khối. Số liệu sẽ nói rõ hơn mọi lý thuyết.
●●● Cài phiên bản cache-oblivious của một thuật toán quen thuộc	[8]	Ví dụ: chuyển vị ma trận bằng chia để trị; so với bản ngây thơ trên ma trận lớn.
●●● Phân tích bài thuê hay mua và chứng minh tỉ lệ 2 là tối ưu	Bài kinh điển	Dùng lập luận đối thủ: đối thủ chọn thời điểm dừng ngay sau khi bạn quyết định mua.

Bài	Nguồn	Điều cần nhận ra (tiếp)
●●● Đo tăng tốc thực tế của một chương trình song song theo số lõi	Bài tập thực nghiệm	Vẽ đường tăng tốc và ước lượng tỉ lệ phần tuần tự; đối chiếu với ba lý do ở mục 5.
●●● So sánh một hệ thống xử lý dữ liệu song song với một chương trình đơn luồng viết cẩn thận	[9]	Chọn một bài đồ thị vừa phải; kết quả thường gây ngạc nhiên và là bài học đáng nhớ nhất chương.

Kết chương và đọc tiếp

Phần D khép lại với một thông điệp thống nhất: mọi kết luận về hiệu năng đều là kết luận *trong một mô hình*. Chương D/23 nói về giới hạn do bản chất bài toán; chương này nói về giới hạn chứng minh được trong mô hình hạn chế, và về những mô hình khác nơi bảng xếp hạng thuật toán đảo lộn. Ba điều đáng mang theo: cận dưới luôn kèm mô hình và muốn vượt qua thì phải phá đúng giả định; đơn vị chi phí quyết định thuật toán tối ưu; và khi lý thuyết mâu thuẫn thực nghiệm, hãy nghi mô hình trước.

Phần E chuyển hẳn sang thực hành. Bốn chương cuối nói về khoảng cách giữa “tôi biết phải làm gì” và “chương trình chạy đúng, đúng hạn”: cài đặt và gỡ lỗi (E/25), thuật toán trong hệ thống thật (E/26), chiến thuật phỏng vấn và thi đấu (E/27), và cuối cùng là lộ trình luyện tập kèm ngân hàng bài tập để mang theo sau khi gấp sách (E/28).

I Đọc thêm: công trình gốc (chỉ liệt kê nguồn đã qua phản biện; bản tiền ấn được ghi rõ)

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc, đọc phần nào
nền tảng	Backurs & Indyk, “Edit Distance Cannot Be Computed in Strongly Subquadratic Time (Unless SETH Is False)”	SIAM Journal on Computing, 2018 (bản hội nghị STOC 2015)	Đọc phần mở đầu và sơ đồ quy dẫn từ bài vectơ trực giao; đây là ví dụ mẫu mực của cận dưới có điều kiện.
nền tảng	Impagliazzo & Paturi, “On the Complexity of k -SAT”	Journal of Computer and System Sciences, 2001	Nơi ETH và SETH được phát biểu. Đọc để biết chính xác giả thuyết neo nói gì — và không nói gì.
tra khi cần	Gajentaan & Overmars, “On a Class of $O(n^2)$ Problems in Computational Geometry”	Computational Geometry: Theory and Applications, 1995	Danh mục các bài 3SUM-khó trong hình học; tra khi bài của bạn có mùi “ba đối tượng thẳng hàng”.
tra khi cần	Frigo, Leiserson, Prokop, Ramachandran, “Cache-Oblivious Algorithms”	FOCS, 1999	Mô hình chi phí thay thế cho mô hình RAM, dùng cho mục 3 của chương này.
tra khi cần	Borodin & El-Yaniv, <i>Online Computation and Competitive Analysis</i>	Cambridge University Press, 1998	Sách chuẩn cho tỉ lệ cạnh tranh ở mục 4; đọc chương về bài toán thuê hay mua và về bộ nhớ đệm.

Nguồn

- [1] M. Agrawal, N. Kayal, and N. Saxena, “PRIMES Is in P,” *Annals of Mathematics*, vol. 160, no. 2, pp. 781–793, 2004.
- [2] A. Laaksonen. “CSES Problem Set.” (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).
- [3] J. R. Shewchuk, “Adaptive Precision Floating-Point Arithmetic and Fast Robust Geometric Predicates,” *Discrete & Computational Geometry*, vol. 18, pp. 305–363, 1997.
- [4] M. Cygan, F. V. Fomin, Ł. Kowalik, et al., *Parameterized Algorithms*. Springer, 2015.
- [5] R. Impagliazzo and R. Paturi, “On the Complexity of k-SAT,” *Journal of Computer and System Sciences*, vol. 62, no. 2, pp. 367–375, 2001.
- [6] A. Backurs and P. Indyk, “Edit Distance Cannot Be Computed in Strongly Subquadratic Time (Unless SETH Is False),” *SIAM Journal on Computing*, vol. 47, no. 3, 2018, bản hội nghị: STOC 2015.
- [7] A. Gajentaan and M. H. Overmars, “On a Class of $O(n^2)$ Problems in Computational Geometry,” *Computational Geometry: Theory and Applications*, vol. 5, no. 3, pp. 165–185, 1995.
- [8] M. Frigo, C. E. Leiserson, H. Prokop, et al., “Cache-Oblivious Algorithms,” in *Proceedings of the 40th Annual Symposium on Foundations of Computer Science*, 1999, pp. 285–297.
- [9] F. McSherry, M. Isard, and D. G. Murray, “Scalability! But at What COST?” In *15th Workshop on Hot Topics in Operating Systems (HotOS)*, 2015.
- [10] D. D. Sleator and R. E. Tarjan, “Self-Adjusting Binary Search Trees,” *Journal of the ACM*, vol. 32, no. 3, pp. 652–686, 1985.
- [11] N. Alon, Y. Matias, and M. Szegedy, “The Space Complexity of Approximating the Frequency Moments,” *Journal of Computer and System Sciences*, vol. 58, no. 1, pp. 137–147, 1999.
- [12] T. H. Cormen, C. E. Leiserson, R. L. Rivest, et al., *Introduction to Algorithms*, 4th ed. MIT Press, 2022.
- [13] P. Flajolet, É. Fusy, O. Gandouet, et al., “HyperLogLog: The Analysis of a Near-Optimal Cardinality Estimation Algorithm,” in *Conference on Analysis of Algorithms (AofA)*, 2007.

Nguồn

- [1] M. Agrawal, N. Kayal, and N. Saxena, “PRIMES Is in P,” *Annals of Mathematics*, vol. 160, no. 2, pp. 781–793, 2004.
- [2] A. Laaksonen. “CSES Problem Set.” (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).
- [3] J. R. Shewchuk, “Adaptive Precision Floating-Point Arithmetic and Fast Robust Geometric Predicates,” *Discrete & Computational Geometry*, vol. 18, pp. 305–363, 1997.
- [4] M. Cygan, F. V. Fomin, Ł. Kowalik, et al., *Parameterized Algorithms*. Springer, 2015.
- [5] R. Impagliazzo and R. Paturi, “On the Complexity of k-SAT,” *Journal of Computer and System Sciences*, vol. 62, no. 2, pp. 367–375, 2001.
- [6] A. Backurs and P. Indyk, “Edit Distance Cannot Be Computed in Strongly Subquadratic Time (Unless SETH Is False),” *SIAM Journal on Computing*, vol. 47, no. 3, 2018, bản hội nghị: STOC 2015.
- [7] A. Gajentaan and M. H. Overmars, “On a Class of $O(n^2)$ Problems in Computational Geometry,” *Computational Geometry: Theory and Applications*, vol. 5, no. 3, pp. 165–185, 1995.
- [8] M. Frigo, C. E. Leiserson, H. Prokop, et al., “Cache-Oblivious Algorithms,” in *Proceedings of the 40th Annual Symposium on Foundations of Computer Science*, 1999, pp. 285–297.
- [9] F. McSherry, M. Isard, and D. G. Murray, “Scalability! But at What COST?” In *15th Workshop on Hot Topics in Operating Systems (HotOS)*, 2015.
- [10] D. D. Sleator and R. E. Tarjan, “Self-Adjusting Binary Search Trees,” *Journal of the ACM*, vol. 32, no. 3, pp. 652–686, 1985.
- [11] N. Alon, Y. Matias, and M. Szegedy, “The Space Complexity of Approximating the Frequency Moments,” *Journal of Computer and System Sciences*, vol. 58, no. 1, pp. 137–147, 1999.
- [12] T. H. Cormen, C. E. Leiserson, R. L. Rivest, et al., *Introduction to Algorithms*, 4th ed. MIT Press, 2022.
- [13] P. Flajolet, É. Fusy, O. Gandouet, et al., “HyperLogLog: The Analysis of a Near-Optimal Cardinality Estimation Algorithm,” in *Conference on Analysis of Algorithms (AofA)*, 2007.